

FujiNet for the Atari 2600

Theory of Operation

How a cartridge with thirteen address lines, eight data lines and no other signal becomes a network adapter, a boot loader and a text generator for a console with 128 bytes of RAM.

Hardware: Fujiversal-Atari2600 Rev0 · **Cartridge firmware:** fujivcs · **Adapter firmware:** fujiversal-atari2600

About this document

This is a theory of operation for the Atari 2600 FujiNet: the Rev0 cartridge board, the firmware on its two microcontrollers, and the console programs that use it. It is written for an engineer who knows the 6507 and the TIA and wants to know what the cartridge does with the bus it is given, how the two chips divide the work, and what has and has not been proven. It explains mechanisms and the reasons for them; it is not a programming manual (that is the *FujiNet Video Computer System Programmer's Handbook*) and not an assembly guide.

Every address, pin, constant, timing figure and listing was transcribed from the live sources below, not from secondary documentation, and the text says which. Quantitative claims carry a tag: **VERIFIED IN EMULATION**, **HOST TEST**, **VERIFIED IN CAD**,

DESIGN VALUE, UNMEASURED or **NOT YET IMPLEMENTED**. No part of this design has yet run on a physical Atari 2600.

Chapter 32 lists what that leaves unproven, including defects found in the source while this document was written.

Canonical sources

fujinet-firmware	branch <code>2600-experiment</code> (7432186c0, 2026-09-27): <code>pico/atari-2600/</code> cartridge firmware, 6507 clients, MAME model, tests; <code>lib/bus/rs232</code> , <code>lib/hardware</code> , <code>include/pinmap</code> adapter side
fujinet-hardware	branch <code>atari2600-rev0</code> (84a8c09, 2026-09-27): <code>ATARI-2600/Fujiversal-Atari2600/</code> KiCad 10 project, README, BOM
fujinet-config	branch <code>add-atari-2600</code> (b4e3805): <code>atari-2600/</code> CONFIG, 16K
fujinet-battleship	<code>atari2600/</code> (9de5ef6): Battleship client, 16K
MOS 6500 family	datasheet AC characteristics, used only to bound the bus timing

Contents

1	Introduction	3
1.1	What the console gives the cartridge	3
1.2	The cartridge-mailbox family	3
1.3	How to read this document	4
1.4	One transaction, end to end	4
2	System architecture	5
2.1	The physical stack	5
2.2	Five layers	5
2.3	Three programs	5
2.4	Division of labour	6
3	The cartridge edge	8
4	Bus buffering	9
5	The RP2040 as bus server	10
6	The timing budget	12
7	The ESP32-S3 and its peripherals	14
8	Power	16
9	Mechanics	17
10	Status of the hardware	18
10.1	What only hardware can answer	18
11	Two cores	20
11.1	core1: the bus, and only the bus	20
11.2	core0: everything else	20
11.3	What crosses between them	20
12	Serving a read	22
12.1	The lookup	23
13	Capturing a write with no R/W line	24
13.1	Defence one: arm, then commit	24
13.2	Defence two: the arming gate	24
13.3	Defence three: the one rule the program must keep	24
14	The window as served	27
15	The mailbox protocol	28
15.1	Registers	28
15.2	One-shot operations	28
15.3	The TX stream	29
15.4	The status cells	29
15.5	Launching, and the interlock	29
15.6	The reply window and its slices	30
16	The text composer	32
16.1	Six planes	32
16.2	The font	32
16.3	Composing a row, and the generation cell	33
16.4	The blit port	33
17	Boot staging and the swap	34

17.1	Power-up	34
17.2	The push	34
17.3	The swap	35
18	The mappers	36
18.1	Side effects after the read	36
18.2	Watching below A12	36
18.3	Choosing a scheme	36
18.4	What is not served, and why	37
19	The link to the adapter	38
19.1	FujiBus	38
19.2	Budgets	38
19.3	The bootloader doorbell	39
20	Nothing here knows the console	41
20.1	Selecting USB	41
21	The device set	42
22	Media: the image as a stream	43
23	One image, two chips	44
24	What a client is	46
24.1	The fixed tail, and why the cold stub lives in it	46
24.2	Every bank carries its own library	47
25	A transaction from the 6507's side	48
26	The display kernel	50
26.1	Positioning	50
26.2	The frame	50
26.3	The kernel	50
26.4	The pixel that cannot be drawn	51
27	Living in 128 bytes	52
27.1	The static check	52
28	Worked examples	53
28.1	The bring-up's browser	53
28.2	CONFIG	53
28.3	Battleship	54
29	Combat, patched for two consoles	56
29.1	The model	56
29.2	What was changed in the 1977 ROM	56
29.3	Where the network runs	57
29.4	Matchmaking	58
29.5	How it was verified	58
30	The emulator model	61
30.1	The device	61
30.2	The harnesses	61
30.3	The bench	62
31	Host tests and static checks	63
32	What nothing proves yet	64
32.1	The bus loop on silicon	64
32.2	Five findings in the source	64
32.3	The hardware checklist	64
32.4	What would close them	64
A	Mailbox reference tables	65

A.1	Blit transforms	65
A.2	Boot and error codes	65
A.3	Console-side names	66
B	Cartridge schemes	67
C	Rev0 bill of materials, principal parts	68
D	FujiBus quick reference	69
E	Sources and glossary	70

PART I

Overview

What the cartridge is, what the console gives it to work with, and how the work is divided between the console, the cartridge microcontroller and the adapter.

CHAPTER 1

Introduction

From the console's side of the connector, the FujiNet cartridge is 4K of ROM whose contents change. From the network's side it is a FujiNet adapter: a WiFi device that mounts disk images from SD cards and TNFS servers, opens TCP and HTTP connections, keeps the time, and answers the same command set every other FujiNet answers. Between the two sits an RP2040 that serves the 6507 bus one access at a time and an ESP32-S3 that does everything else. This document explains the mechanism end to end: the board, the two firmware images, and the shape of a console program that uses them.

1.1 What the console gives the cartridge

The Atari 2600 cartridge edge carries thirteen address lines, eight data lines, +5 V and ground. Every other signal in the console stays inside it. Table 1 lists what is missing and what each absence costs the design; the rest of this document is largely the set of answers.

R/W	A store and a fetch look identical: an address appears, and that is all. The cartridge cannot be told to write, so it must infer a write from where the address points and recover the data by watching the bus (Chapter 13).
Clock ($\phi 2$)	There is no edge to time against. An access is detected only by the address changing , and the cartridge must settle, decode and drive inside the cycle without knowing where in the cycle it is (Chapter 6).
Chip select beyond A12	The cartridge is selected for the whole of <code>\$1000 – \$1FFF</code> and nothing finer. Every page it wants for itself is a page taken from the program.
Reset	The console's RESET switch is a bit in a RIOT register the cartridge never sees. A console restart begins with whatever bank was last selected still mapped, and all cartridge state intact (Chapter 24).
Interrupts	The 6507 has no NMI, no SO and no SYNC, and nothing in the console drives IRQ. This one is a gift: every bus access comes from the program's own instruction stream, so a static scan of the program can prove things an emulator can only sample (Chapter 27).

Table 1. What the cartridge edge does not carry.

Two more constraints are the console's rather than the connector's. The 6507 can address 8K and the cartridge gets the top 4K of it: sixteen pages of 256 bytes for the program, the text it shows, the replies it reads and the registers it writes. And the console has 128 bytes of RAM, mirrored at `$0100` so that the stack lives in the same 128 bytes. There is no framebuffer: the processor builds every scanline as the beam draws it. On every other machine FujiNet runs on, “draw a string” is a memory write. Here it would be a kernel.

1.2 The cartridge-mailbox family

This is the sixth FujiNet cartridge built on the **mailbox** pattern, after the Odyssey², Bally Astrocade, Arcadia 2001, ColecoVision and Channel F. The pattern is the same on each: the console writes a request into registers the cartridge decodes, the cartridge carries it to the adapter over USB as a FujiBus packet, and the reply is painted into memory the console reads back. The protocol engine, `fujimail.c`, is byte-identical across the 2600, ColecoVision and Channel F ports; what differs per console is how a request gets into it, and that is where the 2600 is hardest. The Astrocade, Arcadia and ColecoVision edges carry no write strobe either, and those ports push both directions through the read path. The 2600 port instead takes real stores, because the 6507 drives the data bus during one and the cartridge can see it. `fuji_mailbox.h:7-25`

1.3 How to read this document

Part II is the board: the edge, the buffers, the two microcontrollers, power and mechanics, and the honest status of a design that exists in CAD. Part III is the cartridge firmware on the RP2040, bottom-up from the bus loop to the USB link. Part IV is the adapter firmware on the ESP32-S3, which turns out to contain nothing specific to this console. Part V is the console side: what a client program is, how it talks to the mailbox, and how it draws. Part VI is verification: what the emulator model and the host tests prove, and what they cannot.

Every figure and constant was transcribed from the sources listed in the colophon, and the source is cited beside it in the form `file:line`. Quantitative claims carry one of five tags:

VERIFIED IN EMULATION — observed in the MAME cartridge model against a live `fujinet-pc`.

HOST TEST — proven by a host-side C test of the shared decode or codec.

VERIFIED IN CAD — checked in KiCad (ERC, DRC, netlist audit).

DESIGN VALUE, UNMEASURED — a figure from a datasheet or a comment, not yet measured on this board.

NOT YET IMPLEMENTED — described in a README as required but absent from the source.

IMPORTANT

No part of this design has run on a physical Atari 2600. The board has not been fabricated. Everything above the bus loop has been exercised against a real FujiNet adapter through an emulator model that compiles the cartridge's own sources; the bus loop itself has not. Chapter 32 lists what that leaves open, including five defects found in the source while this document was being written.

1.4 One transaction, end to end

A console program wants the adapter's network configuration. It opens the cartridge's decode gate with two stores carrying two specific values, then sets three registers — device `$70`, command `$C4`, parameter count 0 — each with a pair of stores: one to arm the register, one to commit the value. It commits a sequence number one higher than the cartridge's acknowledged sequence. The RP2040's bus core, which has been watching the address lines settle and sampling the data lines on the two pages it never drives, has by now queued six events to its other core. That core frames a FujiBus packet, sends it over USB CDC to the ESP32-S3, and waits. The ESP32-S3 decodes the packet, dispatches it to its FUJI device, and answers with a 240-byte reply. The RP2040 copies the reply into the 512-byte window at `$1B00`, writes the reply length, the error code and the link status into the status page, and last of all writes the sequence number into `$1F00`. The console, which has been polling `$1F00`, sees the match and reads the SSID straight out of `$1B00` — and, if it wants it on the screen, streams it byte by byte into a text-row register and asks the cartridge to render it. The program never held the reply in RAM, because it has nowhere to put it. **VERIFIED IN EMULATION** `testrom/fujitest.asm`

CHAPTER 2

System architecture

2.1 The physical stack

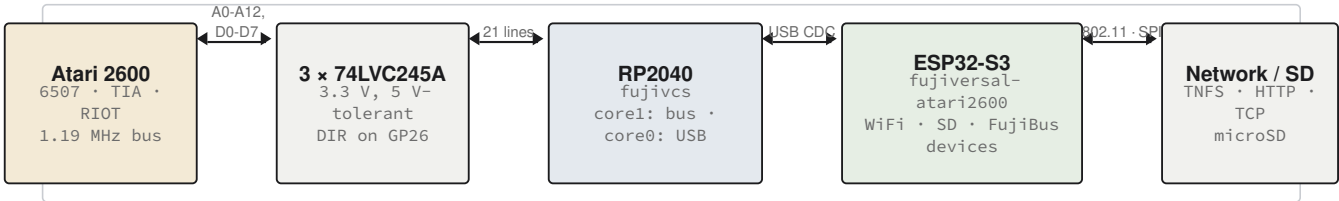


Figure 1. The signal path. Everything left of the buffers is the console’s; everything from the RP2040 rightward is on the cartridge board. The USB link between the two microcontrollers is the only connection between them: there is no UART.

The design is a tandem of two microcontrollers because neither can do the other’s job. The 6507 bus gives a cartridge on the order of 500 ns from the address settling to the data being required at the connector (Chapter 6), with no wait state and no handshake; a missed cycle does not slow the console down, it serves the wrong byte. That rules out any processor that also runs a WiFi stack, TLS, a filesystem and a web server. The RP2040 is given the bus and nothing else, with one of its two cores dedicated to it, running from SRAM with interrupts disabled. The ESP32-S3 is given everything else and never touches the bus. USB joins them, with the ESP32-S3 as host and the RP2040 enumerating as a CDC serial device — the same arrangement as the Intellivision, Astrocade and other Fujiversal boards, so the adapter firmware is shared. `main.c:1-12`

2.2 Five layers

	Layer	Owner	Responsibility
5	Devices	ESP32-S3	FUJL, clock, printer, modem, N: network units; disk images; the FujiNet command set
4	FujiBus	RP2040 ⇄ ESP32-S3	SLIP-framed packets: device, command, parameters, payload, checksum; reply is ACK or NAK plus data
3	Mailbox	RP2040 ⇄ 6507 program	Registers on page \$1D, a TX stream on page \$1E, a painted reply window and status page, text planes
2	Bus server	RP2040 core1	Detect an access from the address changing; drive ROM, or sample a store; bank switching; mappers for booted games
1	Physical bus	Edge + buffers	A0-A12 and D0-D7 at 5 V NMOS levels through 74LVC245A to 3.3 V GPIO; DIR turnaround

Figure 2. The five layers of a transaction. Part II builds layer 1, Part III layers 2–4 from the cartridge side, Part IV layers 4–5 on the adapter, Part V layer 3 from the console side.

2.3 Three programs

fujivcs	RP2040	The cartridge firmware: the bus server on core1, the mailbox service and USB device on core0, the glyph compositor, the mappers, a baked-in boot client. Built by <code>pico/atari-2600/build-cart.sh</code> from <code>pico/atari-2600/firmware/</code> .
---------	--------	--

<code>fujiversal-atari2600</code>	ESP32-S3	A build of <code>fujinet-firmware</code> with <code>BUILD_RS232</code> , the FujiBus transport over USB CDC host, and the pin map in <code>include/pinmap/fujiversal-atari2600.h</code> . Nothing in it knows the console exists.
the client	6507	A 4K–32K image laid out as N banks of 2K plus a fixed 2K half, carrying the four-byte claim <code>FUJI</code> . The firmware ships one baked in; CONFIG and Battleship are the two real ones.

Table 2. The three programs. A booted game is a fourth: an ordinary cartridge image, served by whichever bank-switching scheme it shipped on.

2.4 Division of labour

The guiding rule is that work goes to whichever side has the resources for it, and the console has almost none. Table 3 is the result.

Console (6507, 128 bytes)	A cursor, a few flags, the sequence number it last committed, and the display kernel’s scanline counter. It streams bytes between cartridge pages and never buffers them.
Cartridge (RP2040)	The served window and its decode; four 256-byte path buffers (the working directory, a filter, a copy source, an edit scratch); the reply (up to 1024 bytes); the font and the composed text planes; the Battleship board tables; two 32K image buffers; the mappers; the FujiBus codec and USB device.
Adapter (ESP32-S3)	WiFi, TCP/TLS, HTTP, TNFS, the SD card, disk-image handling, the host and device slot model, configuration, the web UI, and the RP2040’s firmware image for field updates.

Table 3. Who holds what.

PART II

Hardware: Fujiversal-Atari2600 Rev0

The cartridge board as designed: the edge, the bus buffers, the RP2040 and its timing budget, the ESP32-S3 and its peripherals, power, mechanics, and the exact status of a design that has been checked in CAD and never built.

CHAPTER 3

The cartridge edge

The board is a 2×12 gold-finger edge at 2.54 mm pitch, fingers 1.5 × 7.0 mm starting 0.5 mm from the edge, on a 32.4 mm tab with 45° shoulders rising to the full 54.4 mm board width 25.7 mm above the edge. The geometry and the orientation were taken from a working prototype’s gerbers; the component face goes toward the console’s rear.

VERIFIED IN CAD Fujiversal-Atari2600/README.md

1	A7	13	D3
2	A6	14	D4
3	A5	15	D5
4	A4	16	D6
5	A3	17	D7
6	A2	18	A12
7	A1	19	A10
8	A0	20	A11
9	D0	21	A9
10	D1	22	A8
11	D2	23	+5 V
12	GND	24	GND

Table 4. The edge. Pins 1–12 are on the label face (B.Cu, toward the console front), 13–24 on the component face (F.Cu, toward the rear); pin 1 is behind pin 24. bus-buffers.kicad_sch J1

A12 is the only decode the console offers: it is high for every access to `$1000 – $1FFF` and low for everything else, and the cartridge is expected to drive the data bus whenever it is high and the processor is reading. Nothing tells the cartridge which of those two conditions holds. The two schemes that switch banks from addresses **below** A12 — UA on `$0200 – $027F` and FE on `$01FE` — show that a cartridge has always been free to watch the bus when it is not selected, and this one does the same when those boards are emulated (Chapter 18).

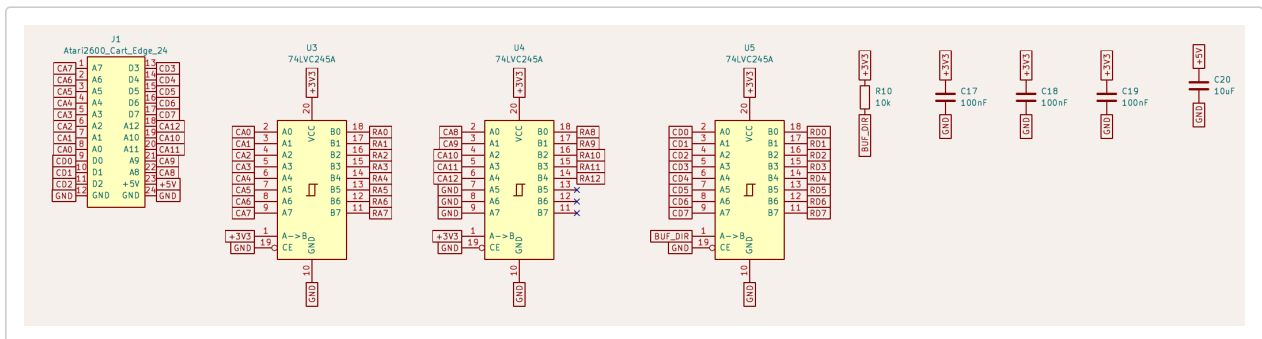


Figure 3. The edge and the three bus buffers, from the Rev0 schematic. U3 and U4 are fixed A→B; U5 carries the data byte and its direction is `BUF_DIR`, pulled up to 3.3 V.

CHAPTER 4

Bus buffering

The RP2040's GPIOs are not 5 V tolerant, and the 2600's bus is an NMOS bus at 5 V. Every one of the 21 lines therefore crosses a 74LVC245A powered at 3.3 V: U3 carries A0–A7 and U4 A8–A12, both wired A→B permanently, and U5 carries D0–D7 with its DIR pin on the RP2040's GP26. `vcs_pins.h:19–27`; README “The RP pin map”

Four properties of the part make the choice. Its inputs are 5 V tolerant, so the console's highs are safe. Its 3.3 V outputs clear the NMOS bus's TTL input threshold of 2.0 V, which is the same margin the PlusCart-Pico already relies on. It has I_{off} circuitry, so an unpowered buffer presents a high impedance to a powered console. And at 3.3 V a single supply is enough, which the dual-supply 74LVC8T245 would not have allowed cleanly: that part references its DIR pin to V_{CCA} , so with the 5 V port as A a 3.3 V GPIO could not reach V_{IH} , and with the 3.3 V port as A the direction sense inverts relative to the firmware's convention. **DESIGN VALUE, UNMEASURED** README “Why 74LVC245A and not the 74LVC8T245”

With the console on U5's A side, **DIR high is console → RP2040** and **DIR low is RP2040 → console**. High is the boot value, and a 10 k pull-up holds it high while the RP2040 is in reset, so a cartridge powered from its USB port with the console switched off cannot back-drive the bus. The firmware sets GP26 high at start-up before it enables anything else, and it drives the data pins at the RP2040's lowest strength, 2 mA, because they only ever face a buffer input and a gentler edge bounds any contention during a direction flip. **AS WRITTEN IN SOURCE** `main.c:41–51`

FINDING 1 · UNVERIFIED ON HARDWARE

The firmware never drives DIR low. `main.c` sets GP26 high at boot, and the `DATA_DRIVE` and `DATA_RELEASE` macros only set and clear the RP2040's own output enables. As written, the cartridge's data byte never reaches the console, and when core1 drives, its 3.3 V outputs fight U5's B-side outputs, which are still pointed at it. Both READMEs record the fix — `DATA_DRIVE` must clear DIR as it sets the output enables, and `DATA_RELEASE` must release the enables first and then set DIR — and both say the cartridge must not be inserted in a powered console with a driving firmware until it is done. The 74LVC245A needs about 5–8 ns to turn around.

NOT YET IMPLEMENTED `vcs_pins.h:59–60`; README-fujinet.md:114–119

The RP2040 as bus server



```
vcs_pins.h:36-44; tools/check_nets.py
```

vcs_pins.h:6-17

DESIGN VALUE, UNMEASURED main.c:34-36; BOM

The RP2040 has no USB connection to the outside world. Its USB pair goes only to the ESP32-S3, so a PC cannot see it as a mass-storage device for a first flash; that is done over the SWD pads (Chapter 23). The two buttons on the RP2040 side are RESET, which resets both chips through a BAT54C common-cathode pair, and BOOTSEL, which holds QSPI_SS low so that a RESET lands the RP2040 in its ROM bootloader.

CHAPTER 6

The timing budget

The cartridge is given no clock, so this chapter has to reason about a cycle it cannot see. Figure 5 draws one, using the MOS 6500 family’s published AC characteristics at 1 MHz as the bound; the 6507 in the 2600 is driven at 1.193182 MHz (the 3.579545 MHz colour clock divided by three), an 838 ns cycle. DESIGN VALUE, UNMEASURED

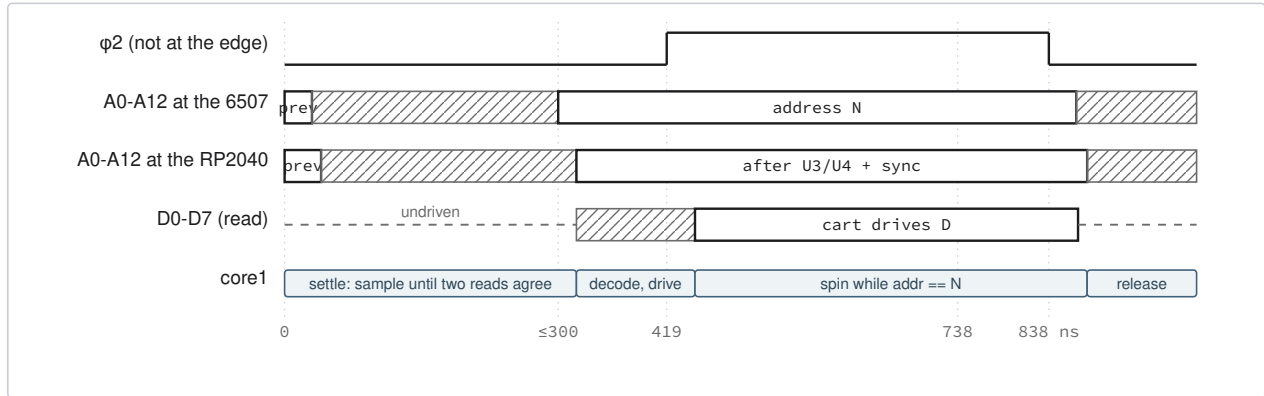


Figure 5. A read cycle as the cartridge experiences it. Time zero is the fall of $\phi 2$ that begins the cycle. The address is guaranteed stable within 300 ns of it (t_{ADS}) and the data must be valid 100 ns before the next fall (t_{DSU}), so the window is about 440 ns at the 6507’s pins in the worst case; the firmware’s working figure is “roughly 500 ns” once the buffers and the RP2040’s two-flop input synchroniser are paid for. Neither has been measured. DESIGN VALUE, UNMEASURED

main.c:28-33

Three things in the figure are the whole design of the bus loop.

The address does not arrive at once. Thirteen lines through two buffers do not change together, and a single sample can catch a mixture of the old address and the new, which on this bus means serving a byte from somewhere else entirely. The loop therefore reads the address until two consecutive reads agree before it believes it. `vcs_cart.c:50-55`

There is no event to wait for. The loop detects an access by the address **changing** from the one it last served. It cannot know whether the current cycle is a fetch or a store, or whether it is early or late in the cycle; it can only respond to the address as fast as possible and keep responding until the address moves on.

The budget is spent in nanoseconds. At 250 MHz a core cycle is 4 ns. The settle test, the page decode, the bank-pointer lookup and the drive are each a handful of instructions, but a single miss in the XIP flash cache is on the order of the whole budget, which is why core1 runs entirely from SRAM (Chapter 11) and why the served image never lives in flash. `boards/fujivcs.cmake:15-19`

A **write** cycle looks different only on the data lines, and only on the two pages the cartridge has declared write-only. Figure 6 is the same cycle for a store to the control page.

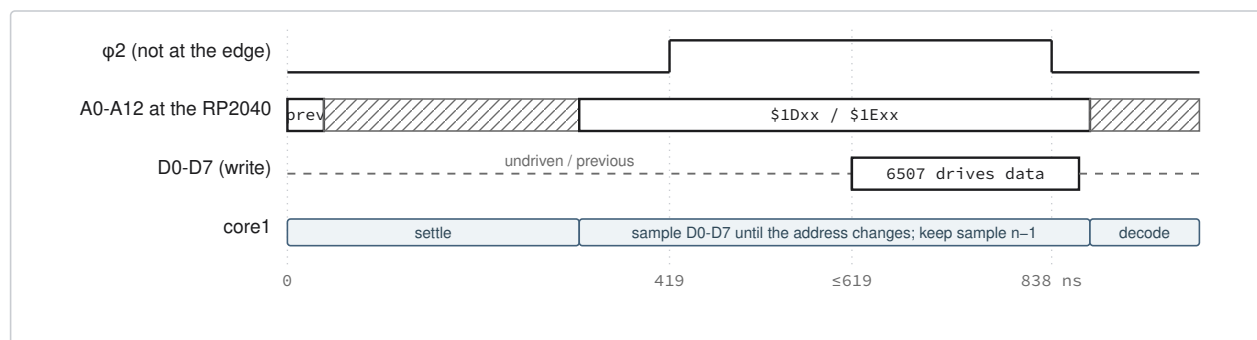


Figure 6. A store to a write-only page. The 6507 drives the data bus from no later than 200 ns after $\phi 2$ rises (t_{MDS}) and holds it at least 30 ns after $\phi 2$ falls (t_{HW}) — the same minimum as the address hold (t_{HA}). The last data sample before the address changes is therefore inside the valid window, and the one before it is the one the cartridge keeps.

DESIGN VALUE, UNMEASURED

The address hold and the write-data hold have the same datasheet minimum. That equality is the margin the technique rests on: the data is still valid when the address begins to change, so the sample taken just before the address moved is good, and keeping the **second-to-last** sample rather than the last one guards against the sampling loop catching the bus as it begins to turn over. How fast the loop samples sets the granularity — two SIO reads and a compare per iteration, a few tens of nanoseconds — and that figure, like every other in this chapter, is a design value.

NOTE

The buffers add about 3 ns each way, and a direction turnaround on U5 about 5–8 ns. These are small against the budget, but the turnaround happens on every transition between a cycle the cartridge drives and one it does not, and the firmware does not yet perform it at all (Finding 1). The hardware README's checklist asks for the address-to-data delay to be scoped at the edge against the 6507's read window before the board is trusted.

CHAPTER 7

The ESP32-S3 and its peripherals

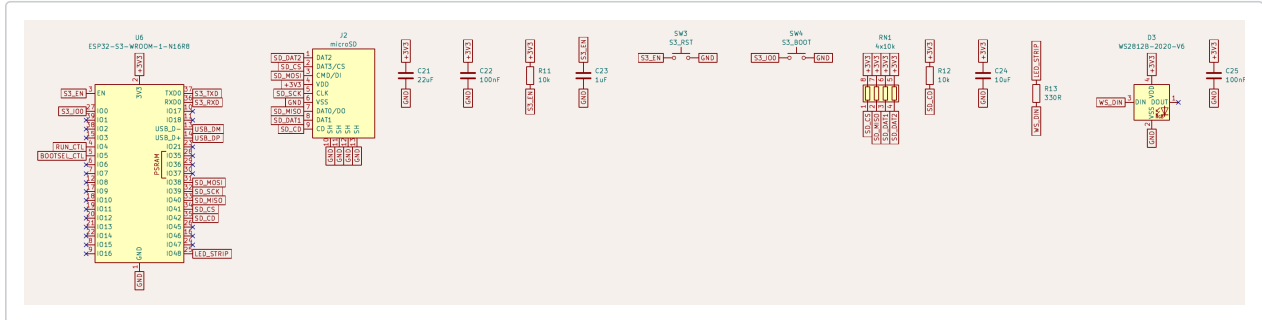


Figure 7. The ESP32-S3-WROOM-1-N16R8, the push-push microSD socket on SPI, the EN and BOOT buttons, and the WS2812B status LED.

The adapter side is an ESP32-S3-WROOM-1-N16R8 module (U6): 16 MB flash and 8 MB octal PSRAM, which is what the FujiNet firmware’s network protocol adapters, TLS and web UI need. Its connections to the rest of the board are few, and all of them are listed in the pin map header the firmware is built against. VERIFIED IN CAD

```
include/pinmap/fujiversal-atari2600.h
```

IO19 / IO20	USB D- / D+	Native USB, acting as host for the RP2040’s CDC device, through 27 Ω series resistors R7 / R8. This is the cartridge link; it has no GPIO of its own in the pin map.
IO38 / IO39 / IO40 / IO41	SD MOSI / SCK / MISO / CS	microSD in SPI mode, with a 4×10 k pull-up array.
IO42	SD card detect	Wired to the socket’s switch with a 10 k pull-up; left unused in the pin map until its polarity is confirmed on hardware.
IO48	LED_STRIP	One WS2812B-2020-V6 through 330 Ω . A 3.3 V-rated part, so no level shifting on its data line. The firmware uses it as a combined status light: white for WiFi up, an orange flicker for bus activity.
IO43 / IO44	UART0 TX / RX	The module’s own console and flashing UART, to the CP2102N. Not the cartridge link.
IO4	RUN_CTL	To the RP2040’s RUN through 1 k: pulse low to reset it.
IO5	BOOTSEL_CTL	To the RP2040’s QSPI_SS through 1 k: hold low across a RUN pulse to force its bootloader.
EN / IO0		The S3 RESET and BOOT buttons; EN is also pulled low by the shared RESET button via D1.

Table 6. The ESP32-S3’s connections. IO26–IO37 are the module’s own flash and PSRAM and strapping pins 0, 3, 45 and 46 are avoided; the SD, LED and recovery pins are the same as the Intellivision board’s, so one firmware image serves both. `fujiversal-atari2600.h:30–66`

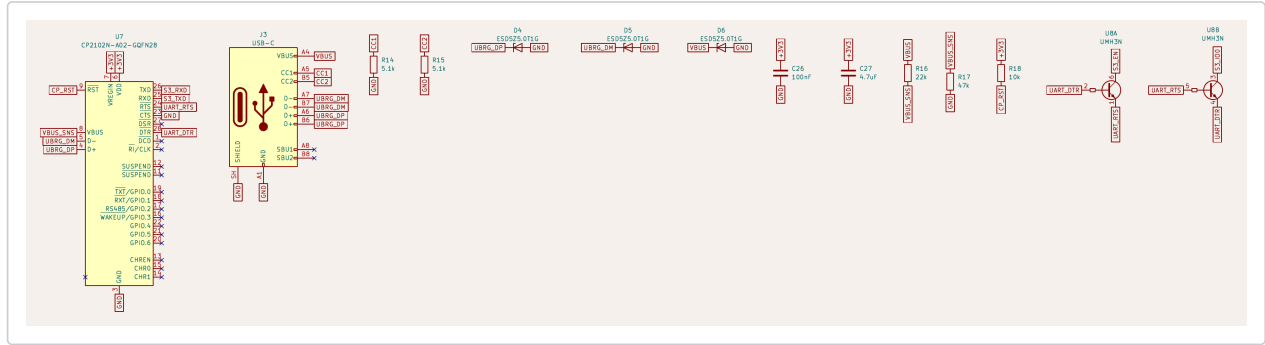


Figure 8. USB-C with a CP2102N bridge and the UMH3N dual-transistor auto-program circuit, DevKitC-1 style, so that `esptool` can reset the S3 into its bootloader without a button press.

USB-C (J3) provides power and the S3's programming port. A CP2102N (U7) bridges it to UART0, and a UMH3N dual transistor (U8) implements the standard DTR/RTS auto-program sequence. Three ESD5Z5.0 diodes protect the USB lines and VBUS; 5.1 k pull-downs on CC1 and CC2 declare a UFP sink. The CP2102N's VBUS-sense pin is fed through a 22 k / 47 k divider so that the bridge enumerates only when a host is actually present.

VERIFIED IN CAD `usb-uart.kicad_sch; BOM`

CHAPTER 8

Power

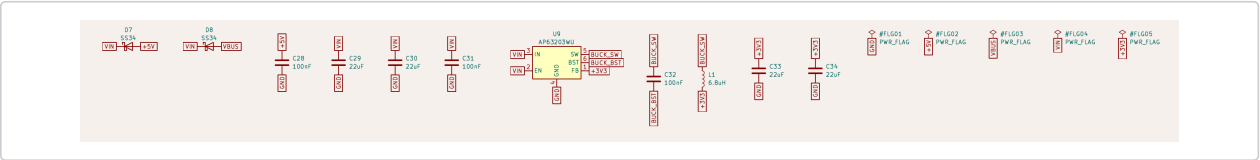


Figure 9. Two SS34 Schottky diodes OR the console’s +5 V and USB VBUS into VIN, which feeds an AP63203 synchronous buck.

The board accepts power from the console’s +5 V (edge pin 23) and from USB VBUS, each through an SS34 Schottky diode so that neither can feed the other. The ORed rail feeds an AP63203WU 3.3 V, 2 A buck converter with a 6.8 μ H inductor, 2 $\times$ 22 μ F on its input and 2 $\times$ 22 μ F on its output. Every other supply on the board — both microcontrollers, the buffers, the SD card, the LED — is on that 3.3 V rail; nothing but the OR diode is connected to the edge’s +5 V. VERIFIED IN CAD power.kicad_sch; README “Power”

Idle, WiFi associated	≈ 60 mA	ESP32-S3 datasheet typicals plus the RP2040 at 250 MHz; the README’s figure
WiFi transmit bursts	up to ≈ 350 mA	ESP32-S3 datasheet peak
Drawn from the console’s +5 V	≈ 250 mA peak	The 3.3 V figures through the buck’s efficiency, with margin

Table 7. The power budget as estimated in the hardware README. The 2600 supplies its +5 V from its own 7805 fed by a 9 V adapter. None of these numbers has been measured, and the README’s checklist asks for the console’s 5 V to be watched under WiFi load before a 2600 Jr is trusted with the board. DESIGN VALUE, UNMEASURED

The decoupling follows the reference designs: 100 nF on every IOVDD, USB_VDD, ADC_AVDD and DVDD pin of the RP2040; 1 μ F on its regulator’s input and output; 10 μ F bulk on 3.3 V and at the edge’s +5 V; 22 μ F $\times$ 5 on the S3’s supply for transmit bursts; 4.7 μ F on the CP2102N. BOM

CHAPTER 9

Mechanics

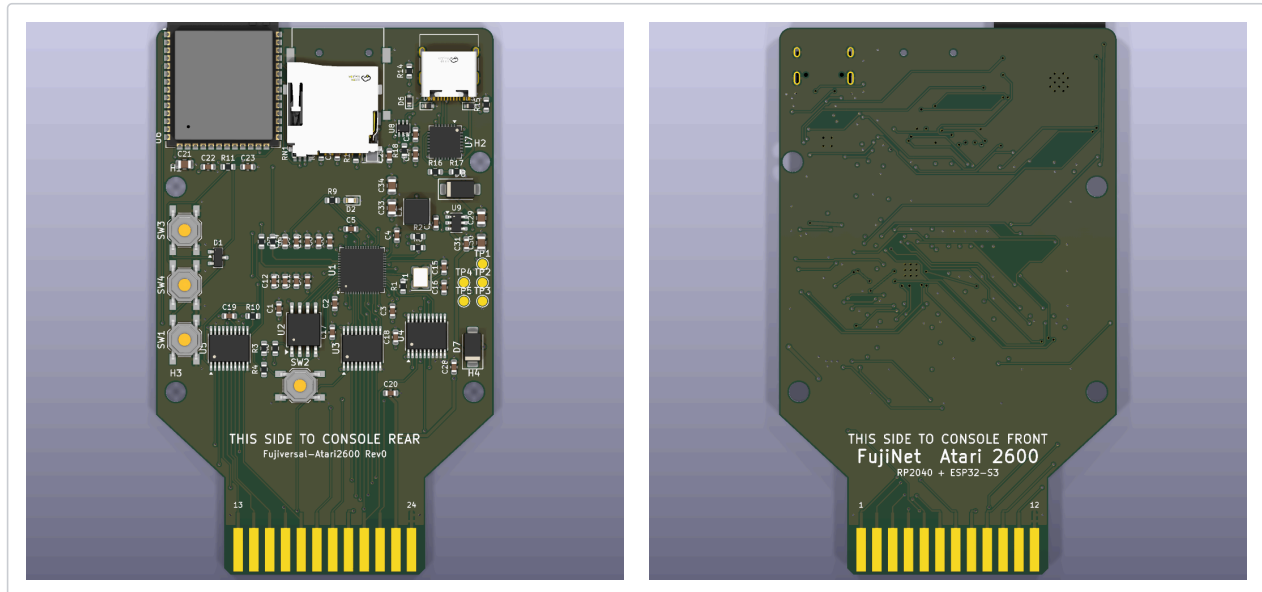


Figure 10. Rev0, component face and label face, rendered from the KiCad project with its bundled 3D models. Every part sits on the component face above the shoulders; the label face carries only the fingers and their necks.

The board is 54.4 × 88 mm, 1.6 mm, four layers, ENIG, with a 30° bevel on the insertion edge. Only the tab and shoulders enter the console. The finger strip is a rule area on both faces — no tracks, vias or pour between the fingers — and each finger continues as a masked 0.5 mm neck to 10 mm from the edge, where the routing picks it up; the inner planes stop 9 mm from the edge, clear of the bevel. The two GND fingers are back to back, so each is stitched to the inner ground plane by a via 1.27 mm outboard of its neck rather than on the neck, where it would short the finger behind it. USB-C and the microSD slot exit the top edge, and the S3’s antenna overhangs it by 6.4 mm into a pocket in the shell. Four M3 holes take a printed two-piece clamshell, 58 × 71 × 10 mm, over the full-width part of the board. **VERIFIED IN CAD** README “Mechanics”, “Routing notes”; case/case-spec.md

The RP2040 is rotated so that A0–A9 leave its south face directly above U3, which sits directly above the A0–A7 fingers in the same order; A10–A12 and D0–D2 leave the east face toward U4; D3–D7 the north face; USB and QSPI the west face toward the S3 and the flash. U5 sits over the data fingers. The intent is the shortest possible bus, and it is the reason the layout could be autorouted to completion. README “How the bus is ordered”

CHAPTER 10

Status of the hardware

Rev0 was generated on 2026-09-27 from `tools/design.py` and `tools/placement.py`: schematic, four-layer layout, 100% routed, and fabrication outputs exported for JLCPCB (gerbers, BOM and CPL; 78 assembled parts, all on the component face). **The board has not been built or tested.** Table 8 is the verification that has been done, all of it in CAD. README “Status”

ERC, <code>--severity-all</code>	0 violations
DRC, <code>--schematic-parity</code>	0 errors, 0 unconnected, 0 parity. 81 warnings, all silkscreen cosmetics
<code>tools/check_nets.py</code>	355 of 355 checks pass. It reads <code>vcs_pins.h</code> , <code>fujivcs.cmake</code> , <code>fujivcs.h</code> and <code>fujiversal-atari2600.h</code> directly and traces every edge pin through its buffer to the right GPIO; it also checks DIR, the output enables, the unused-input ties, the QSPI flash, the USB link, RUN/BOOTSEL, SD, LED and UART. A mutation test (D0 from GP15 to GP16, SD CS from 41 to 42) fails 9 checks, as it should
3D	Every placed part has a bundled model
Firmware	<code>build-cart.sh</code> builds <code>fujivcs.uf2</code> and <code>checksram.py</code> confirms core1 is SRAM-resident; the <code>fujiversal-atari2600</code> PlatformIO environment builds with the updated pin map (with a temporary web-UI config copied from the Astrocade board, which <code>2600-experiment</code> does not yet carry)
Shell	Both halves render in OpenSCAD as 2-manifold
Parts	Every LCSC code resolves to the part named and all are in stock

Table 8. What has been verified on Rev0, and how. VERIFIED IN CAD

10.1 What only hardware can answer

The README ends with a checklist of what CAD cannot verify. It is reproduced here because it is the shortest honest statement of the board’s status, and because Part VI depends on it.

1. **DIR in firmware.** `DATA_DRIVE` must clear GP26 (Finding 1). Until then, never insert the cartridge in a powered console with a firmware that drives.
2. **Fit in a real 2600** — heavy sixer, four-switch, Jr and 7800: the fingers must seat and the shell must clear the slot surround.
3. **Console 5 V under WiFi bursts** at the edge pin, especially on a Jr.
4. **Timing on a scope:** address to data at the edge against the 6507’s read window. Budget about 500 ns; the buffers add about 3 ns each way plus the DIR turnaround.
5. **Back-power:** with USB-C power and the console off, the console must stay dead (DIR high, and the buffers’ I_{off}).
6. **RESET** must bring up the browser; **BOOTSEL + RESET** must enumerate the RP2040 (VID 0x2E8A) on the S3.
7. **microSD card-detect polarity** on IO42, then set `PIN_CARD_DETECT`.
8. **JLC placement rotations:** check the preview against the pin-1 marks.
9. **WiFi RSSI** inside the shell.

PART III

Software: the cartridge firmware

`fujivcs`, the RP2040 firmware: how one core serves the 6507 bus and the other runs the mailbox, how a read is served and a store recovered, the window and its protocol, the glyph compositor, boot staging, the mappers, and the USB link to the adapter.

CHAPTER 11

Two cores

The RP2040 has two Cortex-M0+ cores, and the firmware gives one of them to the bus and nothing else. The split is the first design decision in the cartridge and every other one follows from it. `fuji_cart.h:1-20`

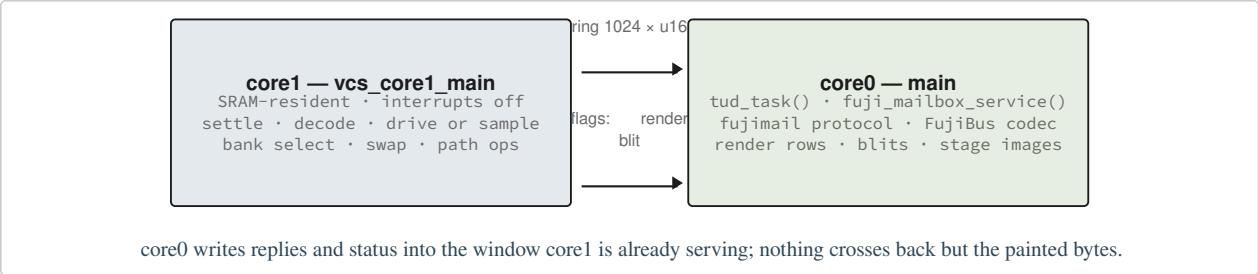


Figure 11. The two cores. core1 produces events into a single-producer, single-consumer ring and raises two flags; core0 consumes them. Replies need no bus-loop involvement at all: the mailbox is painted into memory core1 serves.

11.1 core1: the bus, and only the bus

`vcs_core1_main` is compiled with `__not_in_flash_func`, so the loop and everything it calls is placed in SRAM, and its first act is to disable interrupts for the life of the core. The build asserts this: `build-cart.sh` runs `tools/checksram.py` against the ELF and fails if `vcs_core1_main`, `fuji_cart_serve_staged` or `fuji_cart_poke` is not at a `0x2000xxxx` address. The reason is the budget of Chapter 6: a single XIP cache miss is most of it, so the loop can never be allowed to fault to flash, and no interrupt can be allowed to land in a bus cycle. The served image is for the same reason never in flash either; it lives in two 32K SRAM buffers, which is why 32K is the largest cartridge the design serves. [AS WRITTEN IN SOURCE](#) `vcs_cart.c:1-8, 39-47`; `boards/fujivcs.cmake:11-19`; `tools/checksram.py`

11.2 core0: everything else

core0's main loop is two calls: `tud_task()`, which services the TinyUSB device stack, and `fuji_mailbox_service()`, which drains the ring into the protocol engine and then does the deferred work. It is the only core that talks to the ESP32-S3, the only one that runs `fujimail.c`, and the only one that writes glyphs. `main.c:57-64`; `fujinet.c:135-165`

11.3 What crosses between them

Register write (arm + commit)	two ring entries: <code>REGSEL+n</code> , then <code>REGDATA+value</code>	Expanded into the pair <code>fujimail.c</code> already decodes, so that file stays byte-identical to the sibling ports
TX stream byte	one ring entry: <code>DATA+byte</code>	One store, one event
Path buffer emit (<code>PATH_TX</code> , <code>PATH_TXRAW</code>)	one marker entry	core0 expands it into up to 256 <code>DATA</code> events. Doing that on core1 would cost microseconds against an 838 ns budget
Bank select	handled inline on core1	The next fetch may already be from the new bank; it is one pointer assignment

Swap to the staged image	handled inline on core1	Same reason (but see Finding 4)
Text row render (<code>TEND</code>)	a flag	Composing a row writes 36 bytes through a font lookup, far longer than the next access
Blit	a flag plus the transform code	Same; single-slot, no queue
Path buffer edits (<code>PATH_CH</code> , <code>PATH_OP</code>)	handled inline on core1, in <code>vcs_write</code>	Appending is one byte; see Finding 5 for the operations that are not

Table 9. What core1 hands to core0 and what it keeps. `vcs_cart.c:85-130`; `fuji_cart.h:8-20`

The ring holds 1024 16-bit entries. core1 writes `head`, core0 writes `tail`, and neither touches the other’s index; a full ring drops the event and latches an `overflow` flag rather than blocking the bus loop. The size is chosen from the protocol: one transaction is a dozen register writes, each two entries, plus at most a 320-byte stream, and the console cannot start another until it sees the acknowledgement, so the ring only ever has to hold one transaction. `fuji_cart.h:27-48`, `70-96`

FINDING 5 · UNVERIFIED ON HARDWARE

Three path-buffer operations run inline on core1, inside `vcs_write`, between recovering the store and the next settle: `PATH_POP` walks back over the buffer to the previous separator, and `PATH_COMMIT` and `PATH_SEED` copy up to 256 bytes between the selected buffer and the scratch buffer. On a Cortex-M0+ at 250 MHz a 256-byte copy is on the order of a few microseconds; the console’s next access — the fetch of the instruction after the store, which is in the cartridge — arrives about 838 ns after the store. The emulator model runs the same code inside the write handler, instantly, so nothing in Part VI exercises the gap. The `TEND` and blit cases already show the fix: a flag, with the work and the republished length done by core0. **DESIGN VALUE, UNMEASURED**

`vcs_cart.h:371-424`; `vcs_cart.c:85`

CHAPTER 12

Serving a read

Listing 1 is the bus loop in full. Everything the console ever receives from the cartridge comes through its last five lines.

```
for (;;) {
    while ((addr = ADDR_IN) != prev)          /* settle: two reads must agree */
        prev = addr;

    if (!(addr & 0x1000u)) {                    /* A12 low: not ours ... */
        if (fuji_mem.map.watch_low) {          /* ... unless UA or FE is mapped */
            data = 0; dprev = 0;
            while (ADDR_IN == addr) { dprev = data; data = (uint8_t)DATA_IN; }
            vcs_watch(&fuji_mem, (uint16_t)addr, dprev);
            prev = 0xFFFFFFFFu;
        }
        continue;
    }

    if (vcs_tristate((uint16_t)addr)) {        /* $1D or $1E: never drive */
        data = 0; dprev = 0;
        while (ADDR_IN == addr) { dprev = data; data = (uint8_t)DATA_IN; }
        switch (vcs_write(&fuji_mem, (uint16_t)addr, dprev, &ev_a, &ev_b)) {
            /* ... queue, flag, or act inline (Table 10) ... */
        }
        prev = 0xFFFFFFFFu;                    /* force a fresh settle */
        continue;
    }

    DATA_OUT(vcs_read(&fuji_mem, (uint16_t)addr)); /* a page we drive */
    DATA_DRIVE;
    while (ADDR_IN == addr)
        ;
    DATA_RELEASE;
    prev = 0xFFFFFFFFu;
}
```

Listing 1. core1's loop, with the event switch elided. `vcs_cart.c:49-143`

For a selected address on a page the cartridge drives, the sequence is: look the byte up, put it on the data pins, enable the output drivers, spin until the address changes, disable the drivers. `DATA_OUT` is a single store to the SIO `gpio_togl` register of the XOR between the current output word and the new byte masked to the eight data pins, so only those pins change and nothing else on the port is disturbed; `DATA_DRIVE` and `DATA_RELEASE` are single stores to `gpio_oe_set` and `gpio_oe_clr`. The SIO path is used because it is the only one fast enough; the firmware uses no PIO state machine and no DMA on the bus. `vcs_pins.h:48-60`

Setting `prev` to an impossible value after every served access forces the settle test to run again from scratch, so that two consecutive accesses to the **same** address — a read of a hotspot, say, followed by the fetch that returns to it — are not mistaken for one long access.

12.1 The lookup

`vcs_read` is a static inline in a header, and that header is compiled three times: by core1, by the MAME cartridge device, and by the host tests. That is deliberate: the decode cannot drift between the thing that is tested and the thing that runs. For a FujiNet client it is two comparisons and an index. `vcs_cart.h:199-226`

```
static inline uint8_t vcs_read_ex(const vcs_mem_t *m, uint16_t a, bool commit)
{
    if (!vcs_selected(a))                return 0xFF;
    if (m->map.kind == VCSMAP_FUJI && vcs_tristate(a))
        return 0xFF;                    /* open bus: not ours to drive */
    if (m->map.kind != VCSMAP_FUJI) {      /* a booted game: its own board */
        int b = vcsmap_serve((vcsmap_t *) &m->map, a, 0xFFu, commit);
        return (b >= 0) ? (uint8_t)b : 0xFFu;
    }
    uint16_t off = vcs_off(a);
    if (off < FN_BANK_SIZE)
        return m->lowbank ? m->lowbank[off] : 0xFFu; /* the banked 2K          */
    return m->win[off];                       /* the fixed half, painted */
}
```

Listing 2. The read decode, shared by core1, the emulator and the host tests. `vcs_cart.h:199-221`

The low 2K comes from `lowbank`, a pointer into the live image; the high 2K comes from `win`, a 4K working copy whose upper half holds the image’s fixed half with the mailbox painted over it. A bank switch is therefore one pointer assignment, and it runs inline on core1 because the switch has to be complete before the console’s next fetch. The first version of the firmware copied 2K into the window instead; it passed every test, because the emulator’s device has all the time in the world, and would have served wrong bytes for microseconds on hardware. Every classic 2600 mapper swaps a pointer, and so does this one now. [AS WRITTEN IN SOURCE](#) `vcs_cart.h:444-468`;

README-fujinet.md:221-226

CHAPTER 13

Capturing a write with no R/W line

Two pages of the window, `$1D00 – $1DFF` and `$1E00 – $1EFF`, are declared **write-only**. The cartridge never drives them. Not driving them is what makes the rest possible: when the 6507 stores to one of those pages it drives the data bus itself, the cartridge’s data pins are inputs, and the bus loop can simply watch.

```
while (ADDR_IN == addr) { dprev = data; data = (uint8_t)DATA_IN; }  
/* ... dprev is the recovered byte ... */
```

Listing 3. The recovery loop: park on the stable address, sample the data lines until the address changes, keep the second-to-last sample. The idiom is PlusROM’s and the Super Chip’s, verbatim. `vcs_cart.c:80–83`; `vcs_cart.h:153–173`

Chapter 6 explained why the second-to-last sample is the right one: the 6507 holds its write data at least as long as it holds the address, so the last sample taken while the address was still stable is inside the data’s valid window, and the one before it is safely clear of the bus beginning to turn over. What the chapter did not say is the consequence that shapes the whole protocol.

IMPORTANT

A **read** of a write-only page is bit-identical, at the connector, to a write of it. The cartridge sees an address on page `$1D`, samples a data bus that nobody is driving, and decodes whatever the floating bus held as the stored byte. Nothing on the console can tell it otherwise. The Channel F cartridge, which can tell a store from a fetch, is spared this class of hazard; this one is not, and three defences follow. `fujimailbox.h:26–36`

13.1 Defence one: arm, then commit

A register is never written with one store. The console stores to `$1D00 + n` to **arm** register `n` — the data byte of that store is ignored — and then stores the value to `$1DFF` to **commit** it. An arm carries no value; a commit with nothing armed is discarded; a commit spends the arm. One stray access to the control page therefore changes nothing: it either arms a register that is never committed, or commits a value into nothing. On the Channel F the same pair is protocol shape; here it is the defence, and the disarm-after-one-use in `fujimail.c` does real work.

HOST TEST `vcs_cart.h:332–338, 432–439`; `fujimailbox.h:37–46`

13.2 Defence two: the arming gate

Nothing on the control page decodes at all until two stores arrive in order carrying two specific values: `$B5` to `$1DFC`, then `$4A` to `$1DFD`. The precedent is the Atari 7800, whose BIOS probes cartridge space at power-on and would trip a hotspot-based protocol; PlusCart carries a gate at `$1FF4` for the same reason. This gate is stronger, because the pages are always tri-stated: a probe reads the floating bus and moves on, and the decode stays dead until an ordered pair no probe produces. Every client opens the gate first; the fixed tail’s cold stub does it too, because bank selection is a control-page operation. The gate closes again when an image without the claim boots.

HOST TEST `vcs_cart.h:307–331`; `fujimailbox.h:620–635`

13.3 Defence three: the one rule the program must keep

The sampling model has an answer for a plain store and for an indexed store, and no answer for anything else. Table 10 is what the cartridge sees for each 6507 access shape that can reach a write port.

STA abs	the data HOST TEST cycle, the CPU driving data throughout
STA abs,X with a page-aligned base	the data HOST TEST dummy read of the unindexed address, then the write. With a base low byte of \$00 the index can never carry, so both cycles land on the same address and the cartridge sees one long parked access: floating bus, then the driven data

INC , DEC , ASL , LSR , ROL , ROR	abs or abs,X	undefined: which value the address change falls after is not knowable without a clock: a read (floating bus), a write of the old value, a write of the new
STA (zp),Y , STA (zp,X)		an array access carrying floating-bus data, then the store indirect store performs a read at the target address first

Table 10. What the write sampler sees for each addressing mode. `test_busio.c:260-338`; `fuji_mailbox.h:661-669`

Only `STA` , `STX` and `STY` may therefore target `$1D00 – $1EFF` , and indirect stores may not appear in a client at all, since a static scan cannot know what a zero-page pointer holds. `tools/checkrom.py` disassembles every bank of every client image and fails the build on either (Chapter 27). This can be a proof rather than a heuristic because the 6507 has no interrupts: every access comes from the program’s own instruction stream.

CHAPTER 14

The window as served

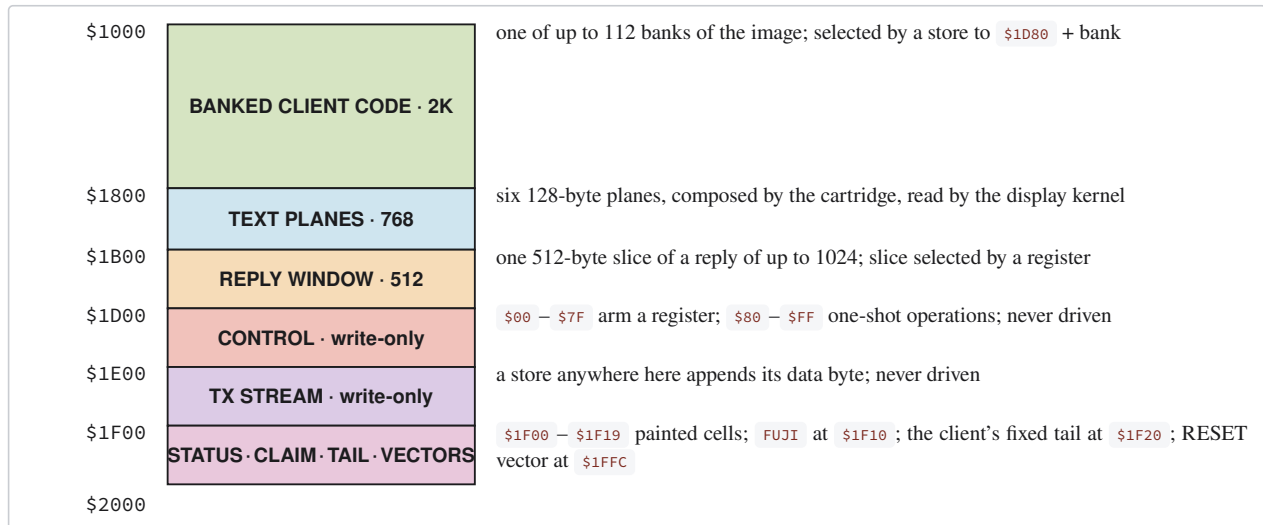


Figure 12. The 4K window as a FujiNet client sees it. The low half is banked; the high half is the mailbox in its entirety, and the one region every bank sees at the same address. `fuji_mailbox.h:53-60`

Sixteen pages is not many, and three decisions in the map each buy a page back.

Two of the three control “pages” are not console addresses. The protocol engine, `fujimail.c`, decodes events by page number: it switches on `offset >> 8` and needs three **distinct** page numbers for register-select, register-data and stream-data. Only one of them, `$1D00`, exists on the bus. `core1` synthesises the other two — `FN_H_REGDATA` is `$2E00` and `FN_H_DATA` is `$2F00`, well outside the window — when it turns a completed arm-and-commit into the pair of events the engine expects, and a TX store into a data event. Two pages of a sixteen-page machine are reclaimed by the choice. `fuji_mailbox.h:70-77`, `215-224`; `vcs_cart.c:86-96`

The reply is 512, not 256 or 1024. 1K would be four of the sixteen pages. 256 would fit in one, but the largest reply the flagship client reads — a Battleship game record — is 509 bytes, so 512 is the size at which it never pages a slice at all. Two slices keep the engine’s 1024-byte maximum for everything else. `fuji_mailbox.h:62-69`

The claim and the vectors live in the fixed half. An image carrying the four bytes `FUJI` at `$1F10` promises that it is a FujiNet client whose mailbox pages are not its own code, and the decode stays live after it boots. A game carries no such promise and the mailbox goes dead for the session. Because the fixed half is never banked, `$1FFC` is reachable from every bank and a console RESET is survivable whatever is mapped low. `fuji_mailbox.h:157-168`; `vcs_cart.h:481-492`

CHAPTER 15

The mailbox protocol

15.1 Registers

Arm with a store to `$1D00 + n`, commit with a store of the value to `$1DFF`. The numbering is the family's, unchanged, so that `fujimail.c` compiles here byte-identically to the ColecoVision and Channel F ports. `fuji_mailbox.h:637-652`

<code>\$00</code>	DEVICE	the FujiBus device id: <code>\$70</code> for the FUJI device, <code>\$71 – \$78</code> for N1:–N8:, <code>\$45</code> for the clock
<code>\$01</code>	CMD	the command byte for that device
<code>\$02</code>	NPARAM	how many parameters head the TX stream
<code>\$05</code>	DATA_RST	any value: rewind the TX stream's write pointer
<code>\$06</code>	RXSLICE	which 512-byte slice of the reply the window shows
<code>\$10</code>	SEQ	nonzero and not equal to ACKSEQ: launch the transaction
<code>\$11</code>	BOOTLOCK	<code>\$B5</code> : arm the ROM swap, honoured only once an image is staged
<code>\$12</code> , <code>\$13</code>	BOOTSEL_1, BOOTSEL_2	<code>\$B5</code> then <code>\$4A</code> as consecutive writes: reboot the RP2040 into its USB bootloader

Table 11. The registers.

15.2 One-shot operations

The upper half of the control page is operations that take effect on the store itself, with the data byte as operand and no commit. None of them disturbs an armed register, so a stray in this half cannot break up a legitimate pair in the other. `vcs_cart.h:340-442`

<code>\$1D80 + b</code>	BANK	serve bank b at <code>\$1000</code> , b up to <code>\$6F</code> ; inline on core1
<code>\$1DF0</code>	TROW	begin composing text row = data; cursor to column 0
<code>\$1DF1</code>	TCHR	append the character; saturates at twelve
<code>\$1DF2</code>	TEND	render the row into the planes on core0; TEXTGEN changes when it has landed
<code>\$1DF3</code>	PATH_CH	append the character to the selected path buffer; saturates at 256
<code>\$1DF4</code>	PATH_OP	a path-buffer operation: RST, POP, TX, TXRAW, POPCH, SEL0–SEL3, COMMIT, SEED (Table 13)
<code>\$1DF5 – \$1DF9</code>	BLIT_SL/SH/DL/DH/CNT	the blit's source offset, destination offset and count
<code>\$1DFA</code>	BLIT_GO	fire the blit with transform = data, on core0; BLITGEN changes when it has landed
<code>\$1DFC</code> , <code>\$1DFD</code>	ARM1, ARM2	<code>\$B5</code> then <code>\$4A</code> : open the decode gate
<code>\$1DFE</code>	SWAP	serve the staged image; inert unless BOOTLOCK has been honoured
<code>\$1DFF</code>	COMMIT	commit the armed register with the data

Table 12. The one-shot operations.

0	RST	empty the selected buffer
1	POP	drop the last path component, keeping the separator: <code>/a/b/c/</code> becomes <code>/a/b/</code> ; <code>/</code> stays <code>/</code>
2	TX	emit the buffer into the TX stream, NUL-padded to exactly 256 bytes
3	TXRAW	emit just its bytes, unpadded (for a URL)

4	POPOCH	drop one character: a keyboard’s backspace
5–8	SEL0–SEL3	subsequent operations act on that buffer; 3 is the edit scratch
9	COMMIT	selected ← scratch: accept an edit
10	SEED	scratch ← selected: begin an edit; cancelling is simply never committing

Table 13. The path-buffer operations. Four 256-byte buffers live in the cartridge because the console has 128 bytes of RAM and a path is 256; the length of the selected buffer is republished at `$1F17` on every change. `fuji_mailbox.h:603–618;`
`vcs_cart.h:363–424`

15.3 The TX stream

Parameters and payload go through page `$1E`. A store anywhere in the page appends its data byte — the address low byte is ignored — which is what makes `STA $1E00,X` legal for any X. The stream has one shape, identical to every sibling port’s:

NPARAM ×	size 1 2 4	value, LE	... then the raw payload
----------	------------	-----------	--------------------------

Each parameter is a size byte, then that many value bytes little-endian; after the last parameter comes the payload as raw bytes. The stream holds 320 bytes, more than any command needs; the two 256-byte path payloads are the largest. `fuji_mailbox.h:654–661;` `fujimail.c:199–210`

15.4 The status cells

<code>\$1F00</code>	ACKSEQ	echoes SEQ when the reply is ready; painted last
<code>\$1F01</code>	STATUS	bit 0: USB link to the adapter is up; bit 1: busy
<code>\$1F02</code>	ERR	transport verdict: 0 ok, 1 no link, 2 timeout, 3 bad frame, 4 too big
<code>\$1F03</code>	REPLY_CMD	<code>\$06</code> if the adapter answered ACK, <code>\$15</code> for NAK
<code>\$1F04</code> – <code>\$1F05</code>	RXLEN	the reply’s length, little-endian
<code>\$1F06</code>	BOOT_STATE	0 idle, 1 transferring, 2 ready, <code>\$80</code> failed
<code>\$1F07</code>	BOOT_PCT	0–100 while an image arrives
<code>\$1F08</code>	BOOT_ERR	1 too big, 2 truncated, 3 no mapper, 4 store busy
<code>\$1F09</code> – <code>\$1F0A</code>	MAGIC	<code>F</code> , <code>N</code> : a cartridge is answering
<code>\$1F0B</code>	PROTO_VER	2
<code>\$1F0C</code>	SLICE_ECHO	the slice number, painted last after a slice change
<code>\$1F0D</code>	TEXTGEN	the last text row rendered, plus a bit that toggles each time
<code>\$1F0E</code>	BANK	the bank currently served
<code>\$1F0F</code>	FLAGS	bit 0: gate open; bit 1: the claim was honoured
<code>\$1F10</code> – <code>\$1F13</code>	CLAIM	<code>FUJI</code> , from the image
<code>\$1F14</code>	HDR	bank count, cell height, layout revision, from the image
<code>\$1F17</code> – <code>\$1F18</code>	PATHLEN	length of the selected path buffer
<code>\$1F19</code>	BLITGEN	incremented after every blit has landed on the planes

Table 14. The status page. Everything here is a plain read. `$1F00` – `$1F0C` are painted by `fujimail.c`; `$1F0D` onward is published by the bus-serving layer, deliberately past the protocol engine’s paint span so that it never clears them. `fuji_mailbox.h:131–160,` `170–195`

15.5 Launching, and the interlock

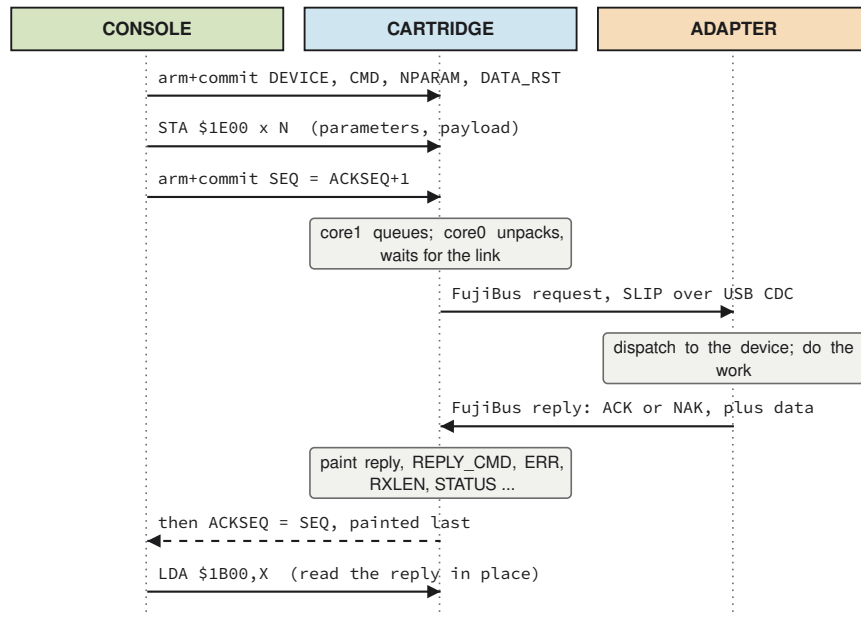
A transaction runs when SEQ is committed with a value that is nonzero and different from the cartridge’s current acknowledged sequence. `core0`’s `run_transaction` unpacks the stream, waits up to 3 s for the USB link if it is down (the console reaches its first transaction long before the ESP32-S3 has enumerated, and the service runs once per SEQ, so a permanent “no link” answer would otherwise be the result), sends the packet, and waits 5 s for the reply — 60 s for MOUNT_IMAGE and COPY_FILE, whose acknowledgements arrive only when the network is done.

Then it paints, in this order: the reply into the window, REPLY_CMD, ERR, RXLEN, STATUS, the slice echo, and **last of all** ACKSEQ. `fujimail.c:6-8, 189-250`

```
poke(FN_R_REPLY_CMD, reply_cmd);
poke(FN_R_ERR, (uint8_t) st);
poke(FN_R_RXLEN_LO, (uint8_t)(rxlen & 0xFF));
poke(FN_R_RXLEN_HI, (uint8_t)(rxlen >> 8));
poke(FN_R_STATUS, port->link_up() ? FN_R_STATUS_LINK : 0);
rxslice = 0;
publish_slice();
/* Published LAST. This single store is the whole interlock: everything
 * the console reads is already in place before it can see the sequence
 * match. */
poke(FN_R_ACKSEQ, seq);
```

Listing 4. The paint order at the end of a transaction. `fujimail.c:237-250`

That single store is the whole interlock between the two sides. There is no interrupt and no handshake line; the console polls `$1F00`, and by the time it can see the match, everything behind it is already readable. A NAK is not a transport error: ERR stays 0 and REPLY_CMD carries `$15`. A reply longer than 1024 bytes is truncated, not refused. **VERIFIED IN EMULATION**



CAUTION

The next sequence number must come from the cartridge's own ACKSEQ cell plus one (wrapping 255 to 1; zero is reserved as "never used"), never from a counter in console RAM. A console RESET restarts the program and re-zeroes its variables, but it does not reset the cartridge: there is no reset line on the connector. A program-local counter would restart at 1, collide with a sequence the cartridge has already answered, and every later transaction would be dropped in silence. Every port in the family has paid for this once; here the RESET-survival test (Chapter 30) checks it. **VERIFIED IN EMULATION** `fujilib.inc:100-116`

15.6 The reply window and its slices

The reply lands in `$1B00 - $1CFF`, 512 bytes at a time, and the cartridge keeps up to 1024 in two slices; RXSLICE selects which the window shows, and after a slice change SLICE_ECHO is painted last so that a program can

wait on it before trusting the bytes. The window is repainted **only** by a SEQ commit or a slice select. Between those it is stable, and that stability is a resource: a program streams bytes straight out of the reply window into the TX page while building its next request, or into a text row, or into a path buffer, with no copy through RAM. On a console with 128 bytes of RAM that is not an optimisation. It is the only way a directory browser fits. `fuij_mailbox.h:200-207`

CHAPTER 16

The text composer

The 2600 has no framebuffer and no character generator. The processor races the beam, and every scanline of the picture is built by hand in the forty-odd cycles the kernel has before the next one begins. On every other FujiNet console “draw a string” is a memory write; here it would be a kernel, and one the console has neither the RAM nor the cycles to feed with glyphs. So the cartridge draws. It keeps the font, renders ASCII into bytes already in the exact shape a 48-pixel player kernel wants, and publishes them in the window. The console streams those bytes into two TIA registers on a cycle-exact schedule (Chapter 26), and that is all it does. `fuji_mailbox.h:86-98`

16.1 Six planes

Text is twelve columns by twenty-one rows. A glyph is three pixels wide and five tall in a four-by-six cell; twelve cells are the 48 pixels that two players with three copies each can cover. Each text row is six scanlines, and each scanline of the 48-pixel block needs six bytes, one per GRP write. The cartridge keeps those six bytes in six separate planes of 128 bytes, indexed by the **absolute scanline**, $Y = \text{row} \times 6 + \text{line}$. `fuji_mailbox.h:99-127`

0	\$1800	1st GRP0	0–1
1	\$1880	1st GRP1	2–3
2	\$1900	2nd GRP0	4–5
3	\$1980	2nd GRP1	6–7
4	\$1A00	3rd GRP0	8–9
5	\$1A80	3rd GRP1	10–11

Table 15. The six text planes.

The 128-byte alignment is load-bearing. Because a plane is 128-aligned and Y is always below 128, the address arithmetic can never carry: `LDA $1800,Y` is always exactly four cycles, never five. A kernel that spends an unpredictable number of cycles does not draw, it tears, so the alignment is the difference between a display and a mess. It also means the kernel needs no zero-page pointers and no per-row setup: Y simply counts from 0 to 125 down the whole screen, and the rows are implicit in the data. Within a byte, bit 7 is the leftmost pixel; the left column of the pair occupies bits 7–5 with bit 4 as its inter-character gap, the right column bits 3–1 with bit 0 as its gap.

7	6	5	4 gap	3	2	1	0 gap
---	---	---	-------	---	---	---	-------

```
for (g = 0; g < FN_T_PLANES; g++) {
    const uint8_t *l = glyph(2*g < len ? text[2*g] : ' ');
    const uint8_t *r = glyph(2*g + 1 < len ? text[2*g + 1] : ' ');
    uint8_t *p = win + (FN_T_PLANE(g) - FN_WINDOW_BASE) + row * FN_T_CELL_H;
    for (s = 0; s < FN_T_CELL_H; s++)
        p[s] = (s < VCS_FONT_INK_H) ? (uint8_t)((l[s] << 5) | (r[s] << 1)) : 0u;
}
```

Listing 5. Composing one text row: for each plane, two glyphs packed into six bytes. The sixth byte of each cell is blank leading. `vcs_render.c:23-46`

16.2 The font

The font is one Python table, `tools/vcsfont.py`, from which the build generates the C header at configure time: 96 glyphs, `$20 – $7F`. Lowercase folds to uppercase and anything outside the table renders as `?`. At three by five

pixels several pairs of characters are identical in most fonts and three were redrawn here: `o` is rounded against `o`’s square, `s` curved against `s`’s flat top, `[` half-width against `c`. That is not a curiosity in a filename browser: `SOAK.BIN` and `50AK.BIN` were the same picture, to the decoder and to a person. The only remaining collision is `?` against `DEL`, which is deliberate. VERIFIED IN EMULATION `vcs_render.c:9-21`; `README-fujinet.md:91-101`;

`CMakeLists.txt:68-90`

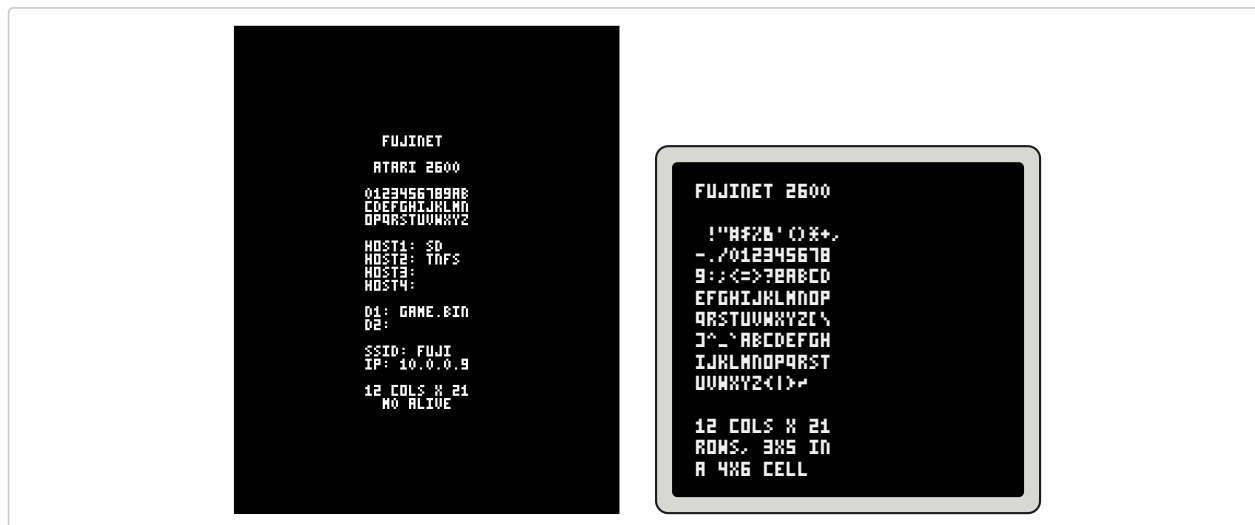


Figure 13. Left: the bring-up’s first screen as the MAME model drew it, the whole font baked into the planes of a plain 4K image with no mailbox. Right: the 12 × 21 grid in the cartridge’s own font, generated from the same table. Every one of the 756 plane bytes of the left image was decoded from the raster and byte-compared against the renderer.

VERIFIED IN EMULATION

16.3 Composing a row, and the generation cell

A program never touches the planes to write text. It stores the row number to `TROW`, the characters to `TCHR`, and anything to `TEND`; `core1` collects the characters in a twelve-byte buffer and raises a flag; `core0` renders the row and then publishes `TEXTGEN` with the row number in its low bits and its top bit toggled. The publish comes **after** the render, so a program that begins composing the next row before `TEXTGEN` changes would hand the render a buffer already being overwritten — on hardware. In the emulator the render runs inside the store, and the cell has already changed by the time the program looks. `fujicart.c:82-94`

16.4 The blit port

Beyond text rows, the cartridge moves and transforms bytes into the planes on request: six stores — a 16-bit source offset into the reply window, a 16-bit destination offset into the planes, a count, and a transform code that fires it — for a screenful the console has neither the RAM nor the raster time to build itself. Twenty transforms exist (Appendix A). They fall into four groups: raw and text copies from the reply; the 10 × 10 game-field renderings that put a Battleship board into text rows; the **playfield** tables, which compose the six per-register tables the Battleship board kernel reads (Chapter 28); and per-cell and path-buffer pokes, which let a list move its cursor or show what has been typed without recomposing a row. `fujimailbox.h:240-470`

CAUTION

A blit is a single-slot request. `core1` latches the transform and raises one flag; there is no queue, so a second blit fired before `core0` has run the first overwrites its arguments and the first is lost — on hardware. `BLITGEN` at `$1F19` is incremented after each blit has landed, and a program waits for it to change before the next. In emulation the blit runs inside the store and the wait is already satisfied. `fujicart.c:96-131`

CHAPTER 17

Boot staging and the swap

17.1 Power-up

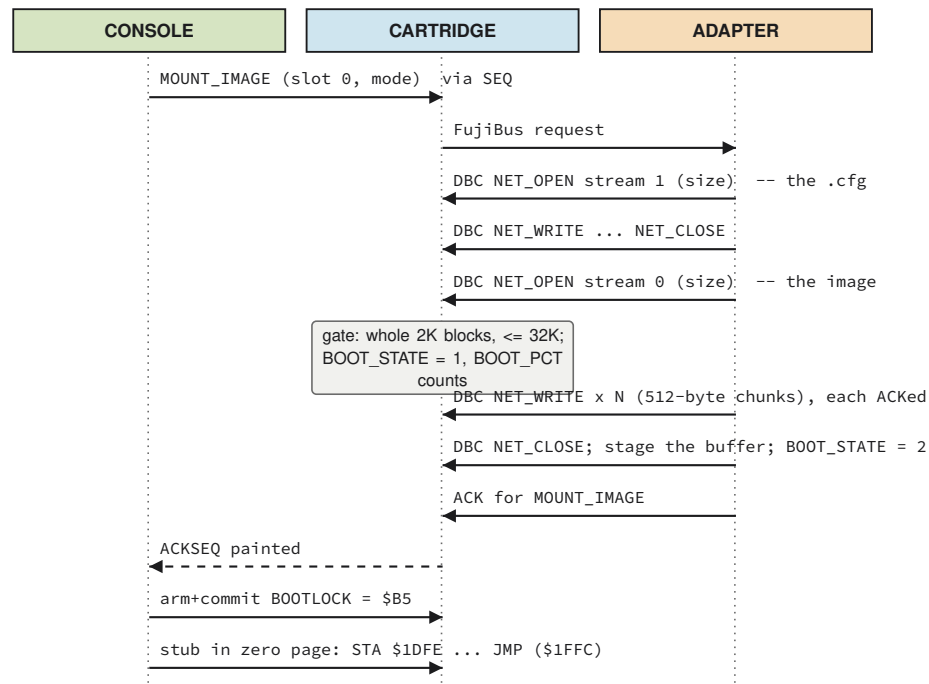
At power-up there is no network and nothing to load a program from, so a client is compiled into the firmware. `build-cart.sh` assembles it, turns the binary into a C array with `tools/mkromh.py`, and builds the UF2 around it; `fuji_config_boot` copies it into image buffer 0 and serves it. Whether the mailbox comes up is decided the same way it is for any image: `vcs_set_image` looks for the claim where the fixed half puts it. The default baked client is `fujidir`, the bring-up's directory browser; CONFIG builds its own `rom.h` for the same slot.

AS WRITTEN IN SOURCE `fujiboot.c`; `build-cart.sh`

17.2 The push

A client boots a game by three FujiNet commands: SET_DEVICE_FULLPATH names the file for device slot 0, MOUNT_IMAGE mounts it, and the swap follows. MOUNT_IMAGE is the one that takes real time, and it is the one command whose reply carries no data at all: while its acknowledgement is outstanding, the adapter streams the image back to the cartridge over the same USB link, addressed to device `$FF` — the “DBC” device, the bus controller itself — as unsolicited NET_OPEN, NET_WRITE and NET_CLOSE frames that the cartridge acknowledges itself. Stream 0 is the image in 512-byte chunks; stream 1, sent first when it exists, is the `.cfg` sibling that names the bank-switching scheme (Chapter 18). **VERIFIED IN EMULATION** `fujimail.c:90-185`; `fujinet.c:50-108`;

`lib/media/rs232/diskTypeROM.cpp`



The size gate runs at NET_OPEN rather than at close, so a file that can never be served is refused before the adapter drags it across the network: a whole number of 2K blocks, at most 32K. A client is $(N+1) \times 2K$ and a game is whatever its board was, and the two are told apart only once the bytes have arrived, by the claim. The image is written into whichever of the two 32K buffers is **not** live, because the client that asked for it may itself

have been booted from the other one and is running out of it. `BOOT_STATE` goes 1 while the image arrives, with `BOOT_PCT` counting, and 2 when it is staged; `$80` with a reason in `BOOT_ERR` if it failed. `fujinet.c:31-46`; `fuji_cart.h:36-44`

17.3 The swap

Two doorbells remain. Committing `$B5` to `BOOTLOCK` arms the swap, and only once an image is staged. A store to `$1DFE` then performs it, and it is inert unless armed. The client copies a short stub into zero-page RAM and jumps to it, because the swap replaces every byte of the window including the code that triggered it:

```

FNSTB:  lda    #0
        sta    GRP0          ; silence the sprites
        sta    GRP1
        sta    AUDV0        ; *** and the sound, or it screams ***
        sta    AUDV1
        sta    PF0
        sta    PF1
        sta    PF2
        sta    WSYNC        ; land on a line boundary
        sta    SWACNT       ; *** RIOT port A back to inputs ***
        sta    SWBCNT       ; *** and port B ***
        lda    #2
        sta    VBLANK       ; blank across the swap
        sta    VSYNC        ; a clean discontinuity, not a rolling frame
        sta    FNRSEL+FH_SWAP ; THE SWAP. The next fetch is at $00xx, in
                                ; RAM with A12 low, so the window may flip
                                ; between this store and it.

        ldx    #$FF
        txs                    ; a clean stack for the new image
        jmp    ($1FFC)        ; the NEW image's own reset vector, exactly
                                ; what the 6507 fetches at power-on

FNSTE:

```

Listing 6. The swap stub, run from zero page. Three stores in it are easy to leave out and each breaks a different thing: the audio volumes, or a tone screams until the game's cold start; and the two RIOT direction registers, because almost no game writes them and a client that left the ports as outputs would brick every game booted after it. `fujilib.inc:254-276`

On `core1` the swap is `fuji_cart_serve_staged`: flip the live-image index, then `vcs_set_image` on the new buffer, which clears the 4K working window, copies the image's last 2K into its upper half, tests for the claim, resets the gate and the armed state, selects bank 0, and initialises the mapper. For a game, the claim is absent, so `mailbox` is false, the decode stays dead for the session, and the image is served by its own board. The `.cfg` hint, if one arrived, is spent on this image and this one only. `fuji_cart.c:60-80`; `vcs_cart.h:472-523`

FINDING 4 · UNVERIFIED ON HARDWARE

The swap runs inline on `core1`, and it is not cheap: a 4K `memset`, a 2K `memcpy` and, for a game, a 1K `memset` of the mapper's RAM. The stub gives it `LDX`, `TXS` and the three bytes of `JMP (abs)` — seven cycles, about 6 μs — before the 6507 reads the new image's reset vector from the cartridge. A Cortex-M0+ at 250 MHz moving 7K through memory routines is in the same range, and is plausibly over it. The stub's own comment covers only the first fetch after the store. The emulator model performs the swap instantly, so nothing in Part VI measures it. The fix is the one banking already uses: prepare the window on `core0` and have `core1` swap a pointer. **DESIGN VALUE, UNMEASURED** `vcs_cart.c:115-117`; `vcs_cart.h:472-479`; `fujilib.inc:268-274`

CHAPTER 18

The mappers

A game that arrives is served by whichever bank-switching scheme it shipped on, and the cartridge has to behave like that board. `vcsmmap.h` is the set, and like the window decode it is a header of static inlines compiled by `core1`, the emulator device and the host tests alike. `vcsmmap.h:1-13`

FLAT	2K or 4K	never; a 2K image is served mirrored
F8	8K, 2 banks	an access to <code>\$1FF8 — \$1FF9</code>
F6	16K, 4 banks	<code>\$1FF6 — \$1FF9</code>
F4	32K, 8 banks	<code>\$1FF4 — \$1FFB</code>
FA	12K, 3 banks	<code>\$1FF8 — \$1FFA</code> ; 256 bytes of RAM written at <code>\$1000</code> , read at <code>\$1100</code>
E0	8K in 1K slices	<code>\$1FE0 — \$1FF7</code> select the three low slices; the top slice is fixed to slice 7, which keeps <code>\$1FFC</code> in place across a RESET
UA	8K, 2 banks	<code>\$0200 — \$027F</code> : below A12 , where the cartridge is not selected and only watches
FE	8K, 2 banks	an access to <code>\$01FE</code> arms it; the bank comes from bit 5 of the byte on the bus in the next access; the first after reset is ignored
CV	2K + 1K RAM	never; RAM read at <code>\$1000</code> , written at <code>\$1400</code>
Super Chip	a flag on F8, F6, F4	128 bytes of RAM read at <code>\$1080 — \$10FF</code> , written at <code>\$1000 — \$107F</code>

Table 16. The schemes served. Every one of them switches on a **read**, because real hardware cannot tell a read from a write and so every classic board keys on the address. `vcsmmap.h:39-52`, `165-290`

18.1 Side effects after the read

A hotspot access returns the **old** bank’s byte. The access that switches the bank still reads what was there before, and the switch applies to the next access. MAME’s own memory system settled this: it installs a read bank and a read **tap** over the same range and runs the tap after the read. The firmware had it inverted until the MAME source decided it, and the difference is invisible until a game reads its own hotspot for data as well as for the side effect — which several do. `vcsmmap_serve` therefore computes the byte first and applies the side effect second, and takes a `commit` flag so that the emulator’s debugger can peek without moving the bank; `core1` always commits.

HOST TEST `vcsmmap.h:24-32`, `213-254`; `README-fujinet.md:213-220`

18.2 Watching below A12

UA and FE are the two boards that make a cartridge watch the bus when it is not selected. When either is mapped, `vcsmmap_init` sets `watch_low`, and `core1`’s A12-low branch — which is otherwise one predicted branch and gone, on every zero-page and TIA access the game makes — samples the cycle the way a write port is sampled and hands it to the mapper. The emulator device installs read-write taps on `$01FE — $01FF` and `$0200 — $027F` for the same purpose. **VERIFIED IN EMULATION** `vcs_cart.c:57-74`; `vcs_cart.h:250-267`; `emu/fujinet.cpp:205-221`

18.3 Choosing a scheme

The claim decides client versus game. For a game the `.cfg` hint wins where it exists; otherwise the scheme comes from the size alone — 2K and 4K FLAT, 8K F8, 12K FA, 16K F6, 32K F4 — plus MAME’s Super Chip heuristic, which is that the first 256 bytes of the image are all the same byte, because a Super Chip cartridge leaves its RAM window as a repeated pattern rather than code. An 8K F8, E0, UA and FE are all 8192 bytes and nothing inside the

file tells them apart, which is why the `.cfg` exists: its content is the scheme's name (`F8` , `F8SC` , `E0` , `UA` , `FE` , `CV` , ...), case-insensitive, with anything after it ignored. **VERIFIED IN EMULATION** `vcsmap.h:295-373`; `vcs_cart.h:494-523`

18.4 What is not served, and why

Only schemes MAME implements are here, because a scheme with no reference cannot be verified against anything; transcribing another emulator's version and testing it against itself would be theatre. That rules out F0, EF, DF, BF and SB. ACE and ELF are ARM-code cartridges, DPC and the Supercharger carry their own coprocessor or BIOS. 3E and 3F need the cartridge to sample the data bus on an access **below** `$1000` — doable with the mailbox's own technique, but it would put the timing-critical sampling into the serve path of every booted game rather than only into the mailbox. `vcsmap.h:14-23`

FINDING 3 · UNVERIFIED ON HARDWARE

The RAM write halves of Super Chip, FA and CV are decoded in `vcsmap_write`, which core1 reaches only through `vcs_write`, which it calls only for the two mailbox pages. A game's store to `$1000 - $107F` (or FA's `$1000 - $10FF`, or CV's `$1400 - $17FF`) takes the driven-read path instead: core1 serves a byte onto a bus the 6507 is driving, and the write never reaches the RAM. The emulator device has a separate write handler and so passes; the PlusCart firmware this port descends from samples these ranges exactly as the mailbox does.

AS WRITTEN IN SOURCE `vcs_cart.c:76, 136`; `vcsmap.h:275-290`

FINDING 2 · UNVERIFIED ON HARDWARE

`vcs_tristate` tests the page and nothing else, and core1 calls it before it knows whether the image is a client or a game. For a booted game, pages `$1D` and `$1E` of its bank are therefore never driven: the game's own code and data there read as open bus. The read decode in `vcs_read_ex` has the mapper test, and the host test that checks a game is given its own `$1D00` goes through `vcs_read_ex`, which is why both it and the emulator pass. Nearly every 4K game has code in those pages. **AS WRITTEN IN SOURCE** `vcs_cart.h:186-193, 208-209`;

`vcs_cart.c:76`; `test_busio.c:370-375`

CHAPTER 19

The link to the adapter

The cartridge is a USB device. TinyUSB runs on core0, the RP2040 enumerates as a CDC-ACM serial port with vendor id `0xCafe` and a product id of `0x4000` plus one bit per compiled interface class, and the ESP32-S3 is the host. Nothing on the cartridge initiates anything; `fujibus_usb.c` reads and writes the CDC pipe TinyUSB gives it. `usb_descriptors.c:28-38`; `fujibus_usb.c:1-5`

```
static void pump(void)
{
    tud_task();
    busy_wait_us(500);
}
```

Listing 7. Every wait on the link pumps the USB stack with a 500 μ s gap. core1’s bus loop shares the SRAM fabric with core0, and calling `tud_task()` back to back turns a brief contention burst into continuous contention for the whole wait. On the Intellivision this was a hard failure; here the bus is roomier, but the gap costs nothing against a multi-second budget. `fujibus_usb.c:35-45`

19.1 FujiBus

What crosses the link is FujiBus, the packet protocol every FujiNet serial transport speaks; the cartridge’s encoder is a line-for-line port of the adapter’s `FujiBusPacket.cpp`. A frame is SLIP-encoded: `$C0` before and after, with `$C0` in the body sent as `$DB $DC` and `$DB` as `$DB $DD`. Inside is a six-byte header, then descriptor bytes, parameters and payload. `fujibus.c:87-92`; `lib/bus/rs232/FujiBusPacket.cpp:17-35`

device	command	length	checksum	descr	params...	payload...
1	1	2 (LE)	1	1		

The length counts the whole decoded packet including the header. The checksum is an 8-bit sum with end-around carry over the whole packet with the checksum byte zeroed. The descriptor’s low three bits say how many parameters follow and how wide they are — 1 to 4 one-byte values, one or two 16-bit values, or one 32-bit value — and bit 7 says another descriptor byte follows, so that widths can be mixed. The reply is a full packet of the same shape whose command byte is ACK (`$06`) or NAK (`$15`). Appendix D has the tables.

The receive buffer is sized for the largest frame that can arrive, which is not a mailbox reply but a DBC push chunk: a six-byte header plus 512 bytes is 518 decoded, and SLIP worst-case doubles that plus the two delimiters, 1088. Undersizing it truncates every ROM push and both ends see a timeout pointing nowhere near the cause. `fujibus_usb.c:14-20`

19.2 Budgets

USB link to come up before the first transaction	3 s	<code>fujimail.c:8, 214-217</code> ; the ESP32-S3 boots far slower than the console
A reply	5 s	<code>fujimail.c:6</code>
A reply to MOUNT_IMAGE or COPY_FILE	60 s	<code>fujimail.c:7, 219-227</code> ; the image is pushed during the wait
A frame write to the pipe	2 s	<code>fujibus_usb.c:48-66</code>
The console’s own timeout in <code>FNGO</code>	$\approx$ 9 s	<code>fujilib.inc:55-56</code> ; deliberately longer than the cartridge’s, so a real timeout is reported as the cartridge’s error

Table 17. Timeouts along the link.

19.3 The bootloader doorbell

Writing `$B5` to register `$12` and then `$4A` to `$13`, as consecutive register writes, calls the RP2040's ROM `reset_usb_boot`; the cartridge reboots into its USB bootloader on the ESP32-S3's host port and a new firmware can be written to it (Chapter 23). There is no acknowledgement to poll. Two registers with two values, like the arming gate, so that no stray can do it. `fujinet.c:111-114`; `fuji_mailbox.h:647-652`

PART IV

Software: the adapter firmware

`fujiversal-atari2600`, the ESP32-S3 build of `fujinet-firmware`: the transport that selects USB, the devices the bus assembles, how a disk image becomes a pushed cartridge, and how one flash image carries both chips' firmware.

CHAPTER 20

Nothing here knows the console

The adapter firmware’s build for this board is `fujiversal-atari2600`. Its platform definition is three lines: the bus is RS232, the platform macro is `BUILD_RS232`, and the pin map is `PINMAP_FUJIVERSAL_ATARI2600`, with USB host and CDC-ACM host support enabled. There is no `BUILD_ATARI2600`. The console’s identity — the 4K window, the mailbox, the text planes, the mappers — lives entirely on the RP2040, and the ESP32-S3 sees only FujiBus packets arriving over a serial channel, exactly as it does for the Odyssey², Astrocade, Arcadia, ColecoVision and Channel F boards. `build-platforms/platformio-fujiversal-atari2600.ini`; `include/pinmap/fujiversal-atari2600.h:3-9`

20.1 Selecting USB

The RS232 bus has three transports: a wired UART, a USB CDC-ACM host, and bus-over-IP for the PC build. Which one a board gets is decided by the **absence** of UART pins in its pin map: if neither `PIN_UART1_RX` nor `PIN_UART2_RX` is defined, `rs232.h` defines `FUJINET_OVER_USB` and the bus’s serial member becomes an `ACMChannel` instead of a `UARTChannel`. The 2600 pin map deliberately defines neither. `lib/bus/rs232/rs232.h:22-28, 127-133`; `fujiversal-atari2600.h:11-12`

The pin map supplies one more thing the bus needs: which USB device to accept. `FN_USB_EXPECTED_VID` is `0xCafe`, TinyUSB’s default and what the cartridge enumerates as; the product id is left unchecked because it varies with the cartridge’s compiled interface set. The VID alone also rejects an RP2040 sitting in its ROM bootloader, which enumerates as `0x2E8A`, so the bus never tries to talk FujiBus to a cartridge waiting to be flashed. `fujiversal-atari2600.h:61-65`; `rs232.cpp:243-258`

`ACMChannel` owns the ESP-IDF USB host stack. At set-up the bus boosts the USB service tasks to priority 24 — just above the WiFi task — so that enumeration wins the CPU during the boot race, and drops them to 20 once WiFi has associated. A reconnect task runs for the life of the program, reopening the device whenever it reappears, so that an RP2040 reset or replug — which a cartridge swap or a bootloader round-trip causes — is noticed without an adapter reboot. `rs232.cpp:27-30, 148-154`; `lib/hardware/ACMChannel.h:65-72`

CHAPTER 21

The device set

`src/main.cpp` assembles the same devices on the bus for every RS232 board. `src/main.cpp:346-366`

\$70	FUJI (<code>rs232Fuji</code>)	the adapter itself: WiFi scan and connect, host slots, directories, device slots, MOUNT_IMAGE, app keys, adapter configuration, the clock-free status calls. <code>GET_ADAPTERCONFIG_EXTENDED</code> (<code>\$C4</code>) is the first command every bring-up client sends
\$45	CLOCK (<code>rs232Clock</code>)	APETime-compatible time and date, extended with additional return formats
\$40	PRINTER (<code>rs232Printer</code>)	a file-backed printer on the SD card or flash
\$50	SERIAL (<code>rs232Modem</code>)	the R: modem emulation
\$71 – \$78	NETWORK (<code>rs232Network</code> , an <code>NDevice</code>)	N1:–N8:, the network units: OPEN a URL on a protocol adapter (HTTP, HTTPS, TCP, TLS, TNFS, and the rest), READ, WRITE, STATUS, CLOSE, and the parser and JSON helpers

Table 18. The devices on the bus. Commands are dispatched through the device classes' tables and the `fujiDevice` mixins; nothing is per-console. `lib/device/rs232/`

From the console program's point of view a FujiNet command is three register values — device, command, parameter count — and a stream. The FUJI device's command set is the same one every FujiNet platform exposes; the *Programmer's Handbook* carries the full card for each. Two that matter for the mechanism are OPEN_DIRECTORY and SET_DEVICE_FULLPATH, which both read **exactly** 256 bytes of payload. A short payload fails the adapter's read, which is why the cartridge's path-buffer emit pads to 256 and why the client has to know the buffer's length to pad the rest itself. `vcs_cart.h:239-248`; `fuji_mailbox.h:175-182`

CHAPTER 22

Media: the image as a stream

`MediaTypeROM`, the RS232 build's media class for a `.bin` cartridge image, is what makes `MOUNT_IMAGE` push. It does not serve sectors; its read and write entry points simply refuse. Instead, on mount it resolves a memory-map sibling — the image's name with `.cfg` (or `.CFG`, for a case-sensitive host) in place of its extension — and exposes two streams: the map, if it exists, and the image. The bus pushes those to device `$FF` as the DBC frames of Chapter 17, the map first, so that the cartridge has the scheme's name in hand before the image's last chunk closes and `vcs_set_image` chooses the mapper. A missing map is not an error; the cartridge falls back to size detection, and the log says so because a wrong map boots to garbage with no other clue. VERIFIED IN EMULATION `lib/media/rs232/diskTypeROM.cpp:31-130`

CHAPTER 23

One image, two chips

A Fujiversal board is flashed as one image. The RP2040’s firmware is built alongside the ESP32’s and embedded in `firmware.bin` as read-only data; the FujiNet flasher writes the ESP32 over the CP2102N exactly as it does for every other board, and the ESP32 then flashes the cartridge itself through the USB link using the RP2040’s PICOBOOT protocol. After a successful flash the ESP32 records the image’s SHA-256 in NVS and on every later boot compares it with what the cartridge reports: a running cartridge with a matching hash is left alone, one in BOOTSEL is always flashed, and one running a different image is asked to reboot into BOOTSEL and reflashed.

docs/fujiversal-flashing.md

Getting the RP2040 into BOOTSEL has four paths, in order of preference, and this board wires three of them:

1200-baud request	Setting the CDC line to 1200 baud is the convention every Pico tool uses for “reboot into the bootloader”. The cartridge firmware must link <code>pico_usb_reset</code> and define <code>PICO_ENABLE_USB_RESET_VIA_BAUD_RATE</code>	cartridge-side; not in the current <code>fujivcs</code> link set NOT YET IMPLEMENTED
RUN and BOOTSEL lines	The ESP32 holds <code>PIN_RP2040_BOOTSEL</code> low across a pulse on <code>PIN_RP2040_RUN</code> . Works with a bricked or hung cartridge firmware	IO5 → QSPI_SS, IO4 → RUN, through 1 k VERIFIED IN CAD
The BOOTSEL button	Hold while pressing RESET; the next ESP32 boot finds it in BOOTSEL	SW2 + SW1 VERIFIED IN CAD
The mailbox doorbell	The console writes <code>\$B5</code> , <code>\$4A</code> to registers <code>\$12</code> , <code>\$13</code> (Chapter 19)	needs a working firmware and a console VERIFIED IN EMULATION

Table 19. Ways into the RP2040’s bootloader.

NOTE

The hardware README records that the PICOBOOT client is not yet on the `2600-experiment` branch, and the flashing guide’s board table lists only the Intellivision, CoCo and MSX boards. On this branch the first flash of a Rev0 board is therefore over the SWD pads with a debug probe, and the embedded-image path is the design intent rather than the current state. **NOT YET IMPLEMENTED** `Fujiversal-Atari2600/README.md “Flashing”`;
docs/fujiversal-flashing.md:8-14

PART V

Software: the console side

What a client program is, how it talks to the mailbox from 128 bytes of RAM, how it draws, and four real programs: the bring-up's browser, CONFIG, Battleship, and the 1977 Combat patched for two consoles over the network.

CHAPTER 24

What a client is

A FujiNet client for this cartridge is an image of N banks of 2K followed by a fixed 2K half, so its size is $(N+1) \times 2048$ bytes. MAME's cartridge loader accepts only certain sizes, so N is 1, 3, 7 or 15: a 4K, 8K, 16K or 32K image. A program that needs five banks pads to seven. Every bank is assembled at `$1000 – $17FF`, the fixed half at `$1800 – $1FFF`, and the parts are concatenated in order. `build.sh:87-121; fuji_cart.h:36-44`

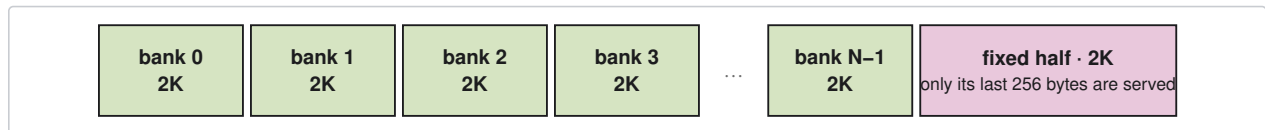


Figure 14. The image. Each bank is entered at `$1000`; the fixed half is the last 2K of the file whatever the size, and the cartridge paints the mailbox over all but its top page.

Of the fixed half only its last 256 bytes reach the console: the rest of its address range is the mailbox. Table 20 is that page.

<code>\$700 – \$70C</code>	<code>\$1F00 – \$1F0C</code>	painted by the cartridge: ACKSEQ, STATUS, ERR and the rest (Chapter 15)
<code>\$70D – \$70F</code>	<code>\$1F0D – \$1F0F</code>	published by the bus layer: TEXTGEN, BANK, FLAGS
<code>\$710 – \$713</code>	<code>\$1F10</code>	the claim, <code>FUJI</code> , stamped by the build at file offset $(\text{size} - 2048) + \$710$
<code>\$714 – \$716</code>	<code>\$1F14</code>	the header: bank count, cell height, layout revision
<code>\$717 – \$719</code>	<code>\$1F17 – \$1F19</code>	published by the bus layer: PATHLEN, BLITGEN
<code>\$720 – \$7FB</code>	<code>\$1F20 – \$1FFB</code>	the fixed tail: 220 bytes of the client's code that every bank must reach at the same address
<code>\$7FC – \$7FF</code>	<code>\$1FFC</code>	the RESET and BRK vectors

Table 20. The top page of the fixed half. `fuji_mailbox.h:131-195; build.sh:48, 81`

24.1 The fixed tail, and why the cold stub lives in it

A bank switch replaces every byte of `$1000 – $17FF`. The store that switches is the last instruction fetched from the old bank, so the instruction after it must be fetched from somewhere that does not change: the tail. And the RESET vector must point into the tail, never into a bank, because the console's RESET switch restarts the 6507 with whatever bank was last selected still mapped. A vector into bank 0 would reset into the wrong code from any other bank. Listing 8 is Battleship's version of both; CONFIG's and Combat's have the same shape.

```
; The trampoline, at FNTAIL ($1F20). A = the bank to enter.
BSGOTO: clc
        adc     #FH_BANK
        tax
        sta     FNRSEL,x      ; the switch; this is the last fetch from the old bank
        ldx     #$FF          ; RESET THE STACK: nothing ever returns through a switch
        txs     ; (fetched from the fixed tail, so it still executes)
        jmp     $1000         ; every bank is entered here

; BSCOLD -- power-on and RESET. Deliberately tiny: only what cannot be
; anywhere else -- the arming pair, because banking is a control-page
; operation, and the bank switch itself.
BSCOLD: sei
```

```

cld
ldx    #$FF
txs
lda    #ENCOLD        ; force a cold entry: the console does not
sta    BSENT          ;   clear RAM on a reset
lda    #FNAM1
sta    FNRSEL+FH_ARM1 ; $B5 ...
lda    #FNAM2
sta    FNRSEL+FH_ARM2 ; ... then $4A: the gate
lda    #BANKLOB
jmp    BSGOTO

ORG    $1FFC
DW     BSCOLD
DW     BSCOLD

```

Listing 8. Battleship’s trampoline and cold stub. The stack reset is not decoration: the switches are taken from inside the per-frame hook, which the display loop reached with a `JSR`, and two bytes leaked per poll until the stack had grown down into the program’s variables. `fujinet-battleship/atari2600/src/bscore.inc:25-80`

Cartridge state survives a RESET: ACKSEQ, the path buffers, the text planes and the reply window are all exactly as they were. A cold stub therefore empties the path buffers explicitly, and derives its first sequence number from ACKSEQ rather than assuming zero (Chapter 15). **VERIFIED IN EMULATION**

24.2 Every bank carries its own library

There is no shared code region but the tail. Each bank includes its own copy of the transport and the display kernel, about 600 bytes, which is what F8 games have always done. Battleship puts the routines every bank calls in the tail — the transport at 151 bytes and the four text primitives at 24 — and generates the equates for them from the tail’s own listing so that no hand-kept list can go stale. A byte added to a shared include costs a byte in every bank that includes it. `fujinet-battleship/atari2600/README.md` “Banks”

CHAPTER 25

A transaction from the 6507's side

The library, `fujilib.inc`, is built on one two-instruction routine, and the whole transport is six steps around it. `testrom/fujilib.inc`

```
; FNRW -- register X = A. The whole transport is built on this.
; `sta FNRSEL,x` is safe for any X: the base low byte is $00, so the index
; can never carry and the dummy read STA abs,X performs lands on the SAME
; address as the write -- one parked access, not two.
FNRW:  sta    FNRSEL,x
        sta    FNCMT
        rts

; FNARM -- open the decode gate: two stores, in order, two specific values.
FNARM:  lda    #FNAM1
        sta    FNRSEL+FH_ARM1
        lda    #FNAM2
        sta    FNRSEL+FH_ARM2
        rts
```

Listing 9. The register write and the gate. `fujilib.inc:31-43`

```
jsr    FNARM            ; open the gate (once)
lda    #FNDEVF          ; 1. the device ($70) ...
sta    FNDEV
lda    #FNCADPX         ; 2. ... the command ($C4, GET_ADAPTERCONFIG_EXTENDED) ...
sta    FNCMD
lda    #0               ; 3. ... and the parameter count
sta    FNNPR
jsr    FNBEG            ; commit all three, rewind TX
                        ; 4. parameters and payload: none here
jsr    FNGO             ; 5. SEQ = ACKSEQ+1, wait; A = 0 if answered
bne    FAILED
jsr    FNACK            ; 6. ACK or NAK?
bne    FAILED
lda    FNRPLY+0         ; the SSID starts at reply offset 0
```

Listing 10. The six steps, for the first command every bring-up client sends. `testrom/fujitest.asm`

```
FNG0:  lda    FNACKS      ; the cartridge's OWN persisted ACKSEQ ...
        clc
        adc    #1        ; ... plus one ...
        bne    FNG01
        lda    #1        ; ... skipping 0, which means "never used"
FNG01: sta    FNSEQ
        ldx    #FR_SEQ
        jsr    FNRW      ; launching it is this single commit
        lda    FNTMO
        sta    FNCNT
FNG02: ldy    #0
FNG03: ldx    #0
FNG04: lda    FNACKS      ; ACKSEQ is published LAST, so a match means
        cmp    FNSEQ      ; everything behind it is already readable
```

```

    beq    FNG05
    dex
    bne    FNG04
    dey
    bne    FNG03
    dec    FNCNT
    bne    FNG02
    lda    #FNEWAIT
    rts
FNG05:  lda    FNERR
    rts
```

Listing 11. Launching and waiting. The timeout is sixteen quanta, about 9 s, deliberately longer than the cartridge’s own 5 s so that a real timeout is reported as the cartridge’s error and not the console’s. In emulation the whole round trip happens inside the commit store and the first comparison matches; on hardware it takes as long as the network does.
fujilib.inc:108-134

Parameters are appended with one store each — a size byte, then the value bytes — by `FNPB` and `FNPW`; a path payload is exactly 256 bytes, NUL-padded, which is what the cartridge’s `PATH_TX` emits in one store. The reply is read in place at `FNRPPLY,X`. Because the window is stable until the next commit, the library streams it rather than copying it: `FNRRPL` renders up to Y bytes of the reply at offset X into text row A, `FNPRPL` appends them to the TX stream of the next request, `FNWRPL` appends them to the selected path buffer. That is how a browser descends into a directory whose name it has never held in RAM:

```
ldx #0 / ldy
#NAMELN / jsr FNWRPL
```

VERIFIED IN EMULATION

fujilib.inc:78-97, 158-178, 302-316, 389-397

CHAPTER 26

The display kernel

`fujidisp.inc` is two routines, shared by every client, and transcribed rather than re-derived from batari Basic's proven kernel. `DINIT` sets the TIA up once; `DLOOP` draws frames forever. `testrom/fujidisp.inc`

26.1 Positioning

Two players with three copies each eight pixels apart (`NUSIZ` = `$03`), so that P0 draws groups 0, 2 and 4 and P1 groups 1, 3 and 5, interleaving into 48 contiguous pixels; vertical delay on both, which is what makes the six-write sequence land in the right copies; then the positioning: sixteen `NOP` s and one three-cycle store so that `RESP0` lands on cycle 38, `RESP1` three cycles later, `HMP0` of `$F0` to pull player 0 right one pixel, and an **early** `HMOVE` on the line after. `fujidisp.inc:20-24, 46-95`

IMPORTANT

The four constants `RNOP` = 16, `R3` = 1, `HM0` = 15, `HM1` = 0 are the coordinates of a one-pixel-wide window. The six copies are 2.67 cycles apart and the kernel's stores are three, so where the block sits decides whether each update lands between copies or inside one, and every wrong setting still looks like text. They were found by decoding the raster back into plane bytes and comparing against the renderer, not by eye.

VERIFIED IN EMULATION

26.2 The frame

VSYNC	3	
VBLANK	37	the client's per-frame hook <code>APPVBL</code> runs here, with the raster nowhere near the text; input scanning belongs here, and so does a transaction, with the caveat that a network round trip is far longer than a frame
pad	33	
text	126	21 rows × 6 lines, the kernel below
pad	33	
overscan	30	
total	262	NTSC, every line timed by <code>WSYNC</code>

Table 21. The frame. `fujidisp.inc:37-42, 98-192`

26.3 The kernel

	tsx				
	stx	SAVSP			
	ldy	#0			
KERN:	sta	WSYNC			
	lda	TP0,y	; 4		
	sta	GRP0	; 3	-> 7	new GRP0 = group 0
	lda	TP1,y	; 4	-> 11	
	sta	GRP1	; 3	-> 14	displays group 0
	lda	TP2,y	; 4	-> 18	
	sta	GRP0	; 3	-> 21	displays group 1
	lda	TP4,y	; 4	-> 25	
	tax		; 2	-> 27	
	txs		; 2	-> 29	park group 4 in S

```

lda    TP3,y      ; 4  -> 33
tax                      ; 2  -> 35
lda    TP5,y      ; 4  -> 39
stx    GRP1       ; 3  -> 42  displays group 2
tsx                      ; 2  -> 44  recover group 4
stx    GRP0       ; 3  -> 47  displays group 3
sta    GRP1       ; 3  -> 50  displays group 4
sty    GRP0       ; 3  -> 53  displays group 5; value unused
iny
cpy    #TLINES
bne    KERN
ldx    SAVSP
txs

```

Listing 12. The text kernel: 53 of the 76 cycles in a line. `fujidisp.inc:144-169`

Three things in it are not style. The stack pointer is a register: the four late writes drift a full cycle across the block, recovering a byte from zero page costs three cycles and `TSX` costs two, and that one cycle is the difference between a schedule that closes and one that does not. The seventh write, `STY GRP0`, exists because with vertical delay six data bytes need seven writes; its value is a don't-care, so the scanline counter serves and no register is spared for it. And nothing between the `TXS` and the restore may touch the stack: no `JSR`, which is safe only because the 6507 has no interrupts. After the last line both players are blanked with three writes, not two — with `VDELP` set, a write to `GRP0` reloads only the delayed `GRP1` and vice versa — and the obvious-looking alternative, toggling `VDELP` off and on, restores the stale delayed registers and redraws the last text line down the blank band. That was one of the three bugs the raster comparison found. `fujidisp.inc:171-179`

26.4 The pixel that cannot be drawn

The 48-pixel block is six player copies and seven writes; the four late ones span 33 pixels, while the distance from the end of group 0 to the start of group 5 is 32. No position serves all 48, so every position loses exactly one pixel, and the least bad one to lose is pixel 7 — bit 0 of plane 0, which the text renderer already spends as column 1's inter-character gap. Text never notices. Anything else drawn through the planes, such as the playing-card transform, has to know, and does: the card bed starts at pixel 2. `fuji_mailbox.h` "FN_CARD_X0"

CHAPTER 27

Living in 128 bytes

The console has 128 bytes of RAM at `$80 – $FF`, and `$0100 – $01FF` mirrors it, so the stack is in the same 128 bytes. The library’s own cells are a sequence number, a timeout counter, the three register values of the current request, a pointer and a count, and two input-scan bytes: fourteen bytes at `$80 – $8E` and `$A8 – $A9`. The swap stub lands at `$80 – $A6` for the few microseconds it runs. Everything else a program might want to hold is in the cartridge: the reply, the working directory, the text, the board. The console holds a cursor and a handful of flags.

`fujinet.inc:103-113; fujiboot.asm:205-207`

27.1 The static check

Because the 6507 has no interrupts, every access a running program makes comes from its own instruction stream, and a scan of the image can prove things an emulator can only sample. `tools/checkrom.py` disassembles every bank — every bank is entered at `$1000` and every bank is code — and fails the build when:

- the image is not $(N+1) \times 2048$ bytes, or not a size MAME will load;
- the claim is missing;
- the RESET vector does not point into the fixed half for a banked image, or anywhere in the window for a flat one;
- any of `INC`, `DEC`, `ASL`, `LSR`, `ROL`, `ROR` with an absolute or absolute,X operand reaches `$1D00 – $1EFF`;
- an indirect `JMP` has a vector on a page boundary (the 6502’s own bug);
- an indirect store, `STA (zp),Y` or `STA (zp,X)`, appears anywhere at all, since the scan cannot know what a zero-page pointer holds.

`tools/checkrom.py:94-160, 206-228, 267-323`

Two traps the build grew checks for are worth recording, because each passed every test before it was caught. `p2bin` emits its whole address range whatever was assembled, so an over-long bank is truncated in silence: the image builds, the size is right, the static check passes, and the instructions past `$17FF` are simply gone; CONFIG’s and Battleship’s build scripts read the assembler’s listing to catch it. And the client equates are hand-mirrored from the header and once drifted by a page; `tools/checkdefs.py` now holds all 39 to it before anything assembles.

`README-fujinet.md:227-237`

CHAPTER 28

Worked examples

28.1 The bring-up’s browser

`fujidir` is a 4K flat image: it mounts host 0, opens `/`, reads an entry per text row, and moves a cursor with the joystick. No name is ever held in RAM; the cursor is an index, and moving it re-reads the listing, which is instant in emulation and about a second a step on hardware. FIRE boots the file under the cursor. It is the client baked into the firmware by default. VERIFIED IN EMULATION `testrom/fujidir.asm:1-17, 216-390`



Figure 15. Left: `fujidir` listing the SD card in the MAME model; the `>` is the cursor, since there is no inverse video. Right: CONFIG’s host-slot screen. VERIFIED IN EMULATION

28.2 CONFIG

CONFIG for this console is a 16K image, seven banks and the fixed half, built in `fujinet-config/atari-2600` with the same assembler and the same checks, and baked into the cartridge in place of the browser. It has feature parity with the Intellivision’s: WiFi set-up, eight host slots, browsing with paging and subfolders and a filter, host-to-host copy, the Game Lobby, adapter information, and mount-and-boot with a progress bar. `fujinet-config` `add-atari-2600:atari-2600/README.md`

0	Host slots	↑↓ move, FIRE mount and browse, SELECT menu: INFO, RENAME, WIFI, LOBBY
1	Browser	↑↓ move, ⇐⇒ page, ⇐ at page 0 goes up, FIRE descend or boot, SELECT menu: UP DIR, FILTER, INFO, COPY, COPY HERE
2	Adapter info	FIRE or SELECT back
3	Keyboard	a 12 × 8 grid, so the cursor position is the character: <code>ch = 32 + row × 12 + col</code> ; SELECT menu: ACCEPT, CANCEL, CLEAR
4	WiFi	↑↓ move, FIRE pick, SELECT skip
5	Boot	progress bar from <code>BOOT_PCT</code> ; FIRE dismisses a failure
6	Copy	FIRE returns early

Table 22. CONFIG’s banks. A joystick, one button, SELECT and RESET are the whole input; the console has no manual, so every action is on a menu that prints itself rather than on a chord.

Almost nothing lives in console RAM: about forty bytes are free, and the working directory, the filter, a pending copy’s source and the text being typed are 256-byte strings in the cartridge. Six mailbox additions were made for it — the four path buffers, the single-character pop, the seed-and-commit edit cycle, and the `PATH` and `TCELL` blits — each because the console could not afford the round trips or the bytes otherwise. Moving a list cursor changes two characters; without a one-cell poke it would cost two transactions per step on the WiFi screen, whose text arrives one scan result at a time. **VERIFIED IN EMULATION**

28.3 Battleship

Battleship is a 16K client in its own repository, playing the family’s `?bin=1&v=2` wire format against the live server at `battleship.carr-designs.com` through N1: over HTTPS, with up to four 10 × 10 boards on screen at once in colour. Text is twelve columns of white; a game board wants colour, and on this console colour comes from the playfield: forty bits across the screen, one colour per scanline. A cell is two playfield bits, so two boards side by side are the whole asymmetric playfield, `PF0`, `PF1` and `PF2` rewritten twice a scanline, and the colour of a mark is decided by **which line of the eight-line cell** is being drawn: a different table in a different colour on each, so that a hit is a red block with a white core, a miss a white dash, and a hull or the cursor a pair of gold brackets.

`fujinet-battleship/atari2600/README.md:1-30, 76-108`

0	<code>bslobby</code> : the cold start and the table list	1016
1	<code>bsgame</code> : the board kernel, the cursor, the cues, a shot	1668
2	<code>bsnet</code> : one request, with the picture up, and back	1993
3	<code>bsmenu</code> : the RESET menu, the help, leaving	1092
4	<code>bsname</code> : the keyboard and the shared username	1201
5	<code>bsplace</code> : the roll, the five ships	2013
6	<code>bscomp</code> : compose the game screen, in passes	2025

Table 23. Battleship’s seven banks of 2048 bytes. `fujinet-battleship/atari2600/README.md` “Banks”

The eighteen tables the kernel reads — three kinds for each of six registers, one entry per cell row — live in the text-plane region, which nothing but a text render ever writes, and the cartridge fills them straight from the reply window with four blit transforms. Doing that on the console would be four hundred reads through a 16-bit cursor in a bank that has not got the bytes, into RAM it has not got at all; the blit costs six stores. The four-seat frame is 262 lines with no slack — 191 lines of kernel in a band of 192 — and `tools/pfcheck.py` reads every cell of every board back from a snapshot and requires the colour the picture says. Every frame of a whole game is held to 262 lines by a harness. **VERIFIED IN EMULATION**

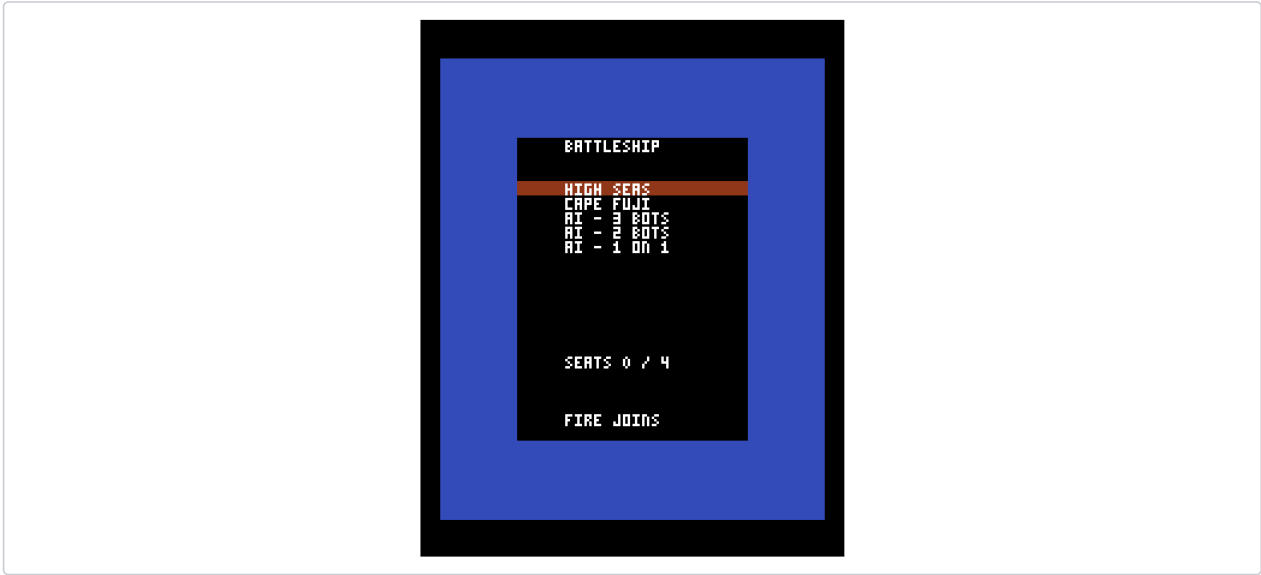


Figure 16. Battleship's lobby, listing the server's tables. VERIFIED IN EMULATION

CHAPTER 29

Combat, patched for two consoles

The last example is the one that tests the design against a program not written for it. *Combat* (Atari, 1977; Larry Wagner and Joe Decuir) is 2K, deterministic, and reads its switches and sticks in exactly four places. `fujinet-2600-combat` is a patch that makes two consoles, each with its own cartridge, play one game of it over the network. It is built from the published Dodgson–Bensema–Williams disassembly, which the repository does not redistribute; the build audits every byte of the result against the pristine ROM assembled at `$1000` and fails on any difference that was not declared. 99 bytes were. **VERIFIED IN EMULATION** `fujinet-2600-combat: README.md; PORTING.md; tools/check_patch.py; tools/patches.py`

29.1 The model

The game is delay-based input lockstep through a relay that interprets nothing. A simulation tick is four video frames, fifteen a second. On each tick every console sends one packed input byte, stamped two ticks ahead, and both consoles apply **both** players’ inputs at the same tick. Local input is delayed by the same two ticks, so the lag is symmetric, about 133 ms, and neither side is authoritative: *Combat* has no random number generator, so two copies given the same inputs in the same order stay identical. `cbdefs.inc:147–154; README.md:58–62`

HELLO	<code>\$01</code>	version 2, TV standard, a 2–8 character name. Sent once by each console
START	<code>\$05</code>	role, seed, delay, variation, the opponent’s name. The console checks that the delay equals its compiled two ticks and falls back to local play otherwise
INPUT	<code>\$06</code>	8 bytes, fixed: the target tick, then a window of three consecutive ticks’ input bytes newest first, a state CRC, and a check byte. The window exists because the transport is not locked to the tick and a tick’s record can go unsent
PEER_LEFT	<code>\$0B</code>	the relay’s notice that the partner dropped

Table 24. The frames the ROM sends or parses. Each is `len, type, payload`; the relay forwards game frames verbatim, padded to 8 bytes, and the console reads only whole frames: `avail AND $F8 . cbdefs.inc:333–354; cbnet.inc:98–180, 243–266; server/combat_relay_server.py:14–28, 83–88`

The input byte is the hardware’s own bits, active low as on the ports: trigger in bit 7, difficulty in bit 6, SELECT in 5, RESET in 4, the stick in the low nibble. RESET and SELECT are ANDed between the two players’ bytes, so either can press them. That AND is also the desync repair: each record carries a CRC over twelve bytes of game state, and a console that sees a mismatch at the tick clears the RESET bit in its own next wire byte — a one-shot synthetic RESET — so that both consoles restart the round at the same tick. A state push would not do, because a tank’s X position is not in RAM at all: it lives in the TIA’s `HMP0 / HMP1 . cbdefs.inc:225–243; cbcrc.inc:394–408, 432–444; PORTING.md $4.26`

29.2 What was changed in the 1977 ROM

<code>\$F014</code>	MLOOP	rewritten	begins with <code>JSR CBSHIM</code> ; if the tick has not advanced the game-logic chain is skipped and only the state loader and the score run, so a stalled frame still draws. <code>INC CLOCK</code> moves behind the gate, so the clock counts ticks that ran
<code>\$F05C</code>	VOUT’s vblank wait	0	<code>LDA INTIM / BNE</code> becomes <code>JSR CBWAIT / NOP / NOP</code> : size-neutral, so the cycle-exact kernel below it does not move
<code>\$F156</code>	VOUT’s final	<code>RTS</code> +3	<code>STA WSYNC</code> , then a bank switch back to the game bank

<code>\$F000</code>	before START	+7	a warm entry: if the session is up, go straight to MLOOP
<code>\$F005</code>	the RAM clear		clears <code>\$80 — \$E5</code> rather than <code>\$80 — \$A2</code> , stopping where the netcode's cells begin
<code>\$F157</code> , <code>\$F192</code> , <code>\$F30E</code> , <code>\$F59C</code>	four reads of <code>SWCHB</code>	0	<code>LDA SWCHB</code> → <code>LDA >CBSWB</code> , the absolute form, so the length and the cycles are unchanged
<code>\$F313</code> , <code>\$F36E</code>	both reads of <code>SWCHA</code>	0	→ <code>LDA >CBJOY</code> : the synthetic stick, both nibbles; the local player's own read is patched too, so that it is delayed like the remote one
<code>\$F3BB</code>	the trigger read	0	<code>LDA INPT4,X</code> → <code>LDA CBTRIG,X</code>
<code>\$F7F4</code>	8 bytes of <code>\$FF</code> filler	8	the tick-phase stepper

Table 25. The declared patches. Seven input sites, each the same length and the same cycles as the original; every other Combat byte keeps its stock offset, rebased from `$F000` to `$1000`. `tools/patches.py:22–252`

The result is an 8K FujiNet client: three 2K banks and the fixed half. The boot bank holds the session and a text screen; the game bank holds START, MLOOP and the game routines; the kernel bank holds VOUT, the score digits, the playfield maps and the network machine. Each bank leaves the other banks' regions empty, so the 1977 code never moves, and the netcode fills the holes the moved regions left. The fixed tail carries the trampoline, the shared transport, the cold stub and the vectors, with 18 bytes to spare. `tools/mkbanks.py:41–49`; `src/cbgame.asm:28–62`; `src/cbtail.asm`; `build/tail.inc`

The RAM budget is the hardest constraint. Combat uses `$80 — $E4` and its stack is `$FA — $FF`, leaving 26 bytes. The input shadows take four of them above the range the game's own clear touches; the netcode's state takes six at `$E6 — $EB`; and `$EC — $F9` is a union, the input rings and the tick counters during play, the transport's cells during the session set-up, because the two never run at once. Combat's own two temporaries are borrowed inside single routines. `cbdefs.inc:6–131`

29.3 Where the network runs

There is no overscan in Combat; the kernel runs to the last line. The network machine therefore runs inside VOUT's vertical-blank wait, as bounded micro-steps dispatched through a jump table, each taken only while the RIOT timer shows at least 448 cycles left against a worst step of about 310. The transport free-runs and is never blocked by the simulation; a WRITE, a STATUS and a READ each cost one frame, so the input exchange cycles at 20 Hz against a 15 Hz tick. Measured over a game, the worst slack was 448 cycles, the average 960, and 0.4% of frames ran no step. **VERIFIED IN EMULATION** `cbdefs.inc:283–308`; `cbnet.inc:24–63`; `PORTING.md:38–54`, 763–765

```

CBWAIT:
    lda    CBENT
    and    #CBE_NET
    beq    CBWSPIN        ; no match: the stock spin, unchanged
CBWLOOP:
    lda    INTIM
    cmp    #CBGATE        ; 8: at least 448 cycles left
    bcc    CBWSPIN        ; through the CARRY: INTIM is unsigned
    jsr    CBNSTEP
    jmp    CBWLOOP
CBWSPIN:
    lda    INTIM
    bne    CBWSPIN
    rts
CBNSTEP:
    ldx    CBNST
    lda    CBJT+1,x        ; the state's handler, through RTS
    pha
    lda    CBJT,x

```

pha
rts

Listing 13. The per-frame hook that replaced Combat’s two-instruction vblank spin. `src/cbnet.inc:24-51`

At each tick boundary `CBSHIM` advances the tick if the last one ran, samples the state CRC, captures the local stick into the ring slot two ticks ahead, and compares the tick the ring has reached against the one the game needs. If the remote input for this tick has not arrived, the frame stalls: it draws, the loader and the score run, the game logic does not. A high nibble in the error cell counts consecutive stalled ticks and saturates at fifteen, about a second; saturation — or a `PEER_LEFT` frame, which saturates it at once — ends the session with “OPPONENT HAS LEFT, PRESS RESET”. A failure during set-up (no cartridge, the open refused, a bad START, a delay mismatch, a 9 s timeout) shows “NO NETWORK, LOCAL PLAY” for 90 frames and then plays plain Combat. `cbinput.inc:75-138`; `cbcap.inc:331-341`; `cbsess.inc:310-324`, 423-439



Figure 17. The session screens, drawn through the cartridge’s text planes by the boot bank; the game bank then runs the 1977 kernel unchanged.

29.4 Matchmaking

The relay listens on TCP port 9600 and auto-pairs the first two idle consoles; the one that waited longest is the host and the left tank. The console reaches it through the N1: device — `OPEN` on `N:TCP://host:9600/` in read-write mode, then `WRITE`, `STATUS` and `READ` — with the URL read from a FujiNet app key that the FujiNet Lobby writes when a player picks the room, and the player’s name from app key 0. The relay registers itself with the Lobby as platform `a2600` with two seats and the client’s TNFS URL, so that choosing the room from any FujiNet machine’s `CONFIG` boots this image and joins. Both players use joystick 1 on their own console; the guest’s byte is swapped into player 1’s slot. `cbsess.inc:29-296`, 373-390; `cbcap.inc:253-275`; `server/combat_relay_server.py:271-299`, 459-528

29.5 How it was verified

A two-console rig: two isolated copies of `fujinet-pc` on bus-over-IP ports 19995 and 19996, the relay on 9600, and two headless MAMEs each running the cartridge model with its own build of the ROM. The verdict requires that the relay paired them, that no CRC mismatch was logged, that both consoles’ state snapshots at a chosen tick are identical, that the game variation actually changed, that more than fifty ticks ran, and that no transport error occurred. `test/run_rig.sh:156-209`

<code>verify-org</code>	the stock ROM rebuilt from the disassembly is byte-identical to the 1977 dump
<code>combat</code>	99 bytes changed, all declared; nothing undeclared
<code>frames</code>	263 lines every frame, through two bank switches a frame
<code>det</code>	1195 frames identical to the unpatched ROM when not in a match
<code>inputs</code>	four port reads in the whole image, all inside <code>CBSHIM</code>
<code>rig</code>	two consoles, 429 and 432 ticks, 0 CRC mismatches, byte-identical zero page at tick 150, through the 8-bit tick wrap
<code>rig-play</code>	581 ticks, identical state at every tick on both consoles
<code>rig-repair</code>	an injected desync is back in agreement within 4–8 ticks
tick wrap	the 8-bit tick wraps cleanly at 45 s

Table 26. Combat’s gates, all run in MAME against `fujinet-pc`. **VERIFIED IN EMULATION** `PORTING.md:738-765`

Not done: no hardware; the Lobby app key for the relay URL is not yet provisioned, so the build-time endpoint is used; Nagle's algorithm is on in the adapter's TCP protocol adapter, which the 20 Hz exchange tolerates but would rather not; the C transliteration of the relay has been checked against the Python one by differential test but not through the rigs; and the PAL build changes only the byte it sends in HELLO, not the kernel. `README.md:121-127`;
`PORTING.md §4.28`

PART VI

Verification, and what is not verified

What the emulator model proves, what the host tests prove, and the precise list of what nothing proves yet: the bus loop on real silicon, and five defects in the source found while this document was written.

CHAPTER 30

The emulator model

Every milestone of the bring-up was passed in MAME, against a live `fujinet-pc`, before any hardware existed, and the design of the verification is as deliberate as the design of the cartridge.

30.1 The device

`emu/fujinet.cpp` is a MAME cartridge device for the `a2600` driver, selected with `-cartslot fujinet`. It is only the port: bytes go into a 4K window and frames go over a TCP socket. Everything that decides what a byte means is the cartridge firmware’s own source, copied into the MAME tree by `emu/apply.sh: fujimail.c, fujibus.c, vcs_render.c, vcs_cart.h` and `vcsmap.h`. The device’s `read()` and `write()` call the same `vcs_read` and `vcs_write` core1 compiles, so the device and the cartridge cannot disagree about which store is a register write or which access is a hotspot. The TCP transport, `emu/fujitcp.c`, carries the same FujiBus frames to `fujinet-pc`’s bus-over-IP listener at `127.0.0.1:9995` by default. `emu/fujinet.h:14-24; emu/fujitcp.c:28-36`

IMPORTANT

What the device does not model is the important thing. MAME hands a cartridge device a clean data byte on a write. The real cartridge has no R/W line and recovers the byte from the second-to-last sample of a bus it is watching; that sampling, the single riskiest thing in the port, never runs in the emulator. `host_test/test_busio.c` is the substitute for the **model** of it (Chapter 31). The loop that performs it on silicon is exercised by nothing. `emu/fujinet.h:28-35`

Two facts about MAME’s memory system shaped the firmware. Its read taps run **after** the read, so a bank-switch hotspot returns the old bank’s byte; the mappers were inverted until the MAME source settled it. And Lua’s `read_u8` is not side-effect free in this build, so a harness that sweeps a banked window trips the game’s own hotspots and has to poll only handler-free offsets. `README-fujinet.md:213-220`

30.2 The harnesses

Each milestone has a Lua script that drives the emulated console and checks the result by decoding memory or the raster, never by eye. `emu/*.lua; README-fujinet.md:121-154`

<code>dispcheck.py</code>	M0	The cart-composed display: decodes a MAME snapshot’s raster back into plane bytes and byte-compares them with the renderer. 756/756. This found three real kernel bugs: a missing seventh <code>GRP</code> write, an end-of-frame drain that toggled VDELP and restored stale registers, and a <code>ldx</code> -versus- <code>tsx</code> cycle
<code>drive.lua</code>	M1	A live <code>GET_ADAPTERCONFIG_EXTENDED</code> reaches the screen: SSID, IP and version
<code>resettest.lua</code>	M1	ACKSEQ survives a console RESET (01 → 02), so the sequence rule holds
<code>boottest.lua</code>	M2	A pushed 4K image with no claim is served byte-identically: 4096/4096 against the file, with the mailbox dead afterwards, as a real game requires
<code>dirtest.lua</code>	M3	The browser navigates to a file by name and boots it, 4096/4096
<code>maptest.lua</code>	M5	F8 hotspots in a booted game switch the right banks
<code>soaktest.lua</code> + <code>tools/soak.sh</code>	M6	Thirteen synthetic images across all nine schemes, Super Chip variants included, each with a <code>.cfg</code> sibling: pushed, booted, the chosen mapper checked, every bank byte-compared. 13/13

<code>cfgtest.lua</code>	M7	CONFIG, from power-on through WiFi, hosts and a subfolder to a booted game
Battleship's <code>drive.lua</code> , <code>frames.lua</code>	M7	A real game against the live server; every frame of a game held to 262 lines

Table 27. The MAME harnesses. All VERIFIED IN EMULATION.

M0 was pulled forward deliberately: every sibling console had a character generator or a framebuffer, so the cartridge-composed display was the one component with no template anywhere in the family. The soak found two things no host test could reach: that an 8K F8, E0, UA and FE cannot be told apart by size, so the `.cfg` had to be read; and that UA and FE switch below A12, where neither core1 nor the device was looking.

30.3 The bench

The long-running `fujinet-pc` is the RS232 build with bus-over-IP listening on `127.0.0.1:9995`; its host slot 0 is an SD directory. The listener accepts one client, so a stray MAME starves the next run and the symptom is a hang rather than an error. MAME must be started from its own tree or the autoboot script is silently ignored, and needs `SDL_VIDEODRIVER=dummy` without a display. `emu/run.sh` handles all three. `README-fujinet.md:195-209`

CHAPTER 31

Host tests and static checks

Four C programs under `firmware/host_test/` compile the shared headers and codec on the build machine with no SDK, and CI runs them on every push as the job “Atari 2600 — bus decode, mappers, glyph compositor, FujiBus codec”. `host_test/Makefile`; `.github/workflows/pico-carts.yml:264-266`

<code>test_busio</code>	The decode and the write-sampling model : which pages are tri-stated; the arming gate; arm and commit; TX appends; the one-shot operations; that the painted window swallows stores; the claim and banking; that <code>STA abs</code> and <code>STA abs,X</code> recover the stored byte through <code>vcs_recover</code> and that an RMW’s three cycles do not; the client-versus-game seam; the path buffers’ pop and pad rules. Its expectations are written from the rules in prose in the headers, never by calling the decoder to see what it does. It tests <code>vcs_read</code> and <code>vcs_write</code> , not the core1 loop
<code>test_vcsmap</code>	Every scheme against a verbatim transcription of MAME’s <code>rom.cpp</code> handlers: 200 000 fuzzed accesses per scheme must agree byte for byte, bank state included
<code>test_render</code>	The compositor against goldens generated from <code>tools/vcsfont.py</code> , the single source of the font
<code>test_fujibus</code>	The FujiBus codec: encode, decode, SLIP, checksum

Table 28. The host tests. All **HOST TEST**.

Three static checks guard the client side. `tools/checkdefs.py` cross-checks the 39 equates in `fujinet.inc` against `fuji_mailbox.h` before anything assembles, because the two are hand-mirrored and once drifted by a page: every register write went to a page that decodes nothing, the client came up, drew a screen, and never armed the mailbox. `tools/checkrom.py` scans every image (Chapter 27). `tools/checksram.py` asserts core1’s residency in SRAM (Chapter 11). `README-fujinet.md:227-232`

CHAPTER 32

What nothing proves yet

This chapter is the one to read before building the board.

32.1 The bus loop on silicon

No test executes `vcs_core1_main`. The emulator never calls it; the host tests call the functions it calls. Everything in Chapter 12 and Chapter 13 about settling, sampling and driving is therefore a design argued from the 6500 datasheet and from the PlusCart firmware the idiom was taken from, and Rev0’s checklist item 4 — address-to-data on a scope against the 6507’s read window — is the first measurement the design needs.

32.2 Five findings in the source

These were found by reading the firmware while writing this document and confirmed against the cited lines. They are the gap between the design as described and the code as written on `2600-experiment` at `7432186c0`, and none can be seen from the emulator or the host tests, because each lives in the one place those do not reach.

1	<code>DATA_DRIVE</code> and <code>DATA_RELEASE</code> never flip DIR, so the cartridge cannot drive the console through U5 and fights its outputs when it tries. Recorded in both READMEs with the fix	Chapter 4
2	<code>vcs_tristate</code> is tested before the mapper is consulted, so a booted game’s <code>\$1D00 – \$1EFF</code> is never driven. The host test that checks the case goes through <code>vcs_read_ex</code> , which has the guard the loop lacks	Chapter 18
3	Super Chip, FA and CV RAM stores take the driven-read path on core1; the write never reaches <code>vcsmem_write</code> , and the cartridge drives against the 6507	Chapter 18
4	The image swap runs 7K of <code>memset</code> / <code>memcpy</code> inline on core1 while the console’s stub reaches the reset vector in about 6 µs	Chapter 17
5	<code>PATH_POP</code> , <code>PATH_COMMIT</code> and <code>PATH_SEED</code> run loops of up to 256 iterations inline on core1 while the next fetch, from the cartridge, is 838 ns away	Chapter 11

Table 29. Findings. Items 2–5 have the same shape: work on core1’s path that the emulator performs instantly and silicon cannot. Items 2 and 3 need a mapper test in the loop; 4 and 5 need the work moved to core0 with a pointer or a flag handed back, as banking, rendering and blits already do.

32.3 The hardware checklist

Chapter 10 reproduced the nine items the Rev0 README says only a built board can answer: DIR in firmware, fit, the console’s 5 V under WiFi load, timing on a scope, back-power, RESET and BOOTSEL behaviour, card-detect polarity, placement rotations, and RSSI in the shell. Items 3 and 4 are measurements; the rest are observations. None has a stand-in.

32.4 What would close them

A built Rev0, a debug probe on the SWD pads for the first flash, a scope on the edge, and a console. The five findings are firmware changes that can be made and tested in the emulator **before** that, with the knowledge that the emulator cannot show whether they were needed — only the board can. That order is the one the rest of the family has followed, and it is the reason this document exists in the form it does: as a statement of intent precise enough to be checked against the silicon when it arrives.

APPENDIX A

Mailbox reference tables

1.1 Blit transforms

Six stores: source offset into the reply window (`$1DF5` , `$1DF6`), destination offset into the text planes (`$1DF7` , `$1DF8`), count (`$1DF9`), then the transform to `$1DFA` , which fires it. What each field means depends on the transform. BLITGEN at `$1F19` increments when it has landed. `fuji_mailbox.h:245-470`

0	RAW	copy <code>cnt</code> bytes from the reply to the planes unchanged
1	TEXT	compose text row <code>dst</code> from up to twelve ASCII bytes at reply offset <code>src</code>
2	FIELD	a 10 × 10 game field at reply offset <code>src</code> (0 sea, 1 hit, 2 miss) into ten text rows from <code>dst</code> , with a row digit in column 0; <code>cnt</code> = cursor cell or <code>\$FF</code>
3	HULLS	<code>cnt</code> ship placements at reply offset <code>src</code> as hull glyphs over the composed board, then paint from row <code>dst</code>
4, 5, 6	SEA, CELL, PAINT	compose a board not in the reply: fill with sea; set <code>board[cnt]</code> to the low byte of <code>src</code> ; paint into ten rows from <code>dst</code>
7	PATH	<code>cnt</code> characters of the selected path buffer from offset <code>src</code> into text row <code>dst</code>
8	TCCELL	one character (low byte of <code>src</code>) into one cell; <code>dst</code> = row × 12 + column
9	CARD	five playing cards from <code>hand[11]</code> at reply offset <code>src</code> across the two text rows from <code>dst</code> ; <code>cnt</code> bit 0 draws the first face down
10	PFCLR	clear the kinds in mask <code>src</code> in playfield slot <code>dst</code>
11	PFIELD	a game field at reply offset <code>src</code> into playfield slot <code>dst</code>
12	PFHULL	<code>cnt</code> placements at reply offset <code>src</code> OR-ed into slot <code>dst</code> 's hull tables
13	PFCELL	one cell <code>cnt</code> of slot <code>dst</code> : set the kinds in mask <code>src</code> , or clear them if bit 7 is set
14	POKE	<code>plane[dst]</code> = the low byte of <code>src</code>
15	PATHPOKE	<code>plane[dst .. dst+cnt]</code> = <code>path[src .. src+cnt]</code> : a block of raw bytes out of a path buffer
16	PFTILE	a packed 20 × 19 tile bitset at reply offset <code>src</code> into the playfield tables; <code>dst</code> = kind mask, <code>cnt</code> = rows (0 = all)
17	PFTCELL	one tile: <code>dst</code> = kind mask, <code>cnt</code> = row, low byte of <code>src</code> = column; a clear bit in it clears
18	PATHTILE	PFTILE out of the selected path buffer: <code>src</code> = byte offset into it
19	MULEMAP	a 225-byte MuleMap at reply offset <code>src</code> into the planes; <code>dst</code> = flags

Table 30. The twenty blit transforms. 0–9 compose text and text-row boards; 10–13 and 16–18 compose the six playfield tables the Battleship and maze kernels read; 14–15 and 19 are raw plane writes.

1.2 Boot and error codes

BOOT_STATE <code>\$1F06</code>	0 / 1 / 2 / <code>\$80</code>	idle / transferring / ready (staged) / failed
BOOT_ERR <code>\$1F08</code>	1 / 2 / 3 / 4	too big (over 32K) / truncated (push aborted) / no mapper (not a whole number of 2K blocks) / store busy
ERR <code>\$1F02</code>	0 / 1 / 2 / 3 / 4	<code>FB_OK</code> / <code>FB_ENOLINK</code> no USB CDC connection / <code>FB_ETIMEOUT</code> / <code>FB_EBADFRAME</code> SLIP, length or checksum failed / <code>FB_ETOOBIG</code>
REPLY_CMD <code>\$1F03</code>	<code>\$06</code> / <code>\$15</code>	ACK / NAK from the adapter; a NAK is not a transport error
STATUS <code>\$1F01</code>	bit 0 / bit 1	link up / busy
FLAGS <code>\$1F0F</code>	bit 0 / bit 1	gate open / claim honoured

Table 31. Status values. `fuji_mailbox.h:679–690`; `fujibus.h:43–49`

1.3 Console-side names

The 6502 clients address the mailbox through hand-mirrored equates in `testrom/fujinet.inc`; `tools/checkdefs.py` holds them to the header.

<code>FNTEXT</code>	<code>\$1800</code>	<code>FNRSEL</code>	<code>\$1D00 + n</code>
<code>FNRPLY</code>	<code>\$1B00</code>	<code>FNCMT</code>	<code>\$1DFF</code>
<code>FNACKS</code>	<code>\$1F00</code>	<code>FNTX</code>	<code>\$1E00</code>
<code>FNSTAT</code>	<code>\$1F01</code>	<code>FR_DEV / FR_CMD / FR_NPAR</code>	<code>\$00 / \$01 / \$02</code>
<code>FNERR</code>	<code>\$1F02</code>	<code>FR_DRST / FR_RXSL</code>	<code>\$05 / \$06</code>
<code>FNRCMD</code>	<code>\$1F03</code>	<code>FR_SEQ / FR_BLK</code>	<code>\$10 / \$11</code>
<code>FNXL0 / FNXL1</code>	<code>\$1F04 / \$1F05</code>	<code>FH_BANK</code>	<code>\$80</code>
<code>FNBST / FNBPC / FNBER</code>	<code>\$1F06 – \$1F08</code>	<code>FH_TROW / FH_TCHR / FH_TEND</code>	<code>\$F0 / \$F1 / \$F2</code>
<code>FNMA0 / FNMA1</code>	<code>\$1F09 / \$1F0A</code>	<code>FH_PATHC / FH_PATHO</code>	<code>\$F3 / \$F4</code>
<code>FNPER / FNSECH</code>	<code>\$1F0B / \$1F0C</code>	<code>FH_ARM1 / FH_ARM2</code>	<code>\$FC / \$FD</code>
<code>FNBTXG / FNBBNK / FNBLG</code>	<code>\$1F0D – \$1F0F</code>	<code>FH_SWAP</code>	<code>\$FE</code>
<code>FNPLNL / FNPLNH</code>	<code>\$1F17 / \$1F18</code>	<code>FNAM1 / FNAM2</code>	<code>\$B5 / \$4A</code>

Table 32. The client equates. `testrom/fujinet.inc`

APPENDIX B

Cartridge schemes

.cfg			
FLAT , 2K , 4K	FLAT	2048, 4096	yes
F8 , F8SC	F8 (+ Super Chip)	8192	yes, F8; SC by the first-256-bytes heuristic
F6 , F6SC	F6	16384	yes
F4 , F4SC	F4	32768	yes
FA	FA	12288	yes
E0	E0	8192	no: needs the .cfg
UA	UA	8192	no: needs the .cfg
FE	FE	8192	no: needs the .cfg
CV	CV	2048 + 1K RAM	no: needs the .cfg

Table 33. Scheme names accepted from a `.cfg` sibling, case-insensitive, with leading blanks and trailing text ignored. The hint is spent on the next image and that one only. Not served: F0, EF, DF, BF, SB (no MAME reference), ACE and ELF (ARM code), DPC and Supercharger (coprocessor or BIOS), 3E and 3F (data sampling below \$10000). `vcsmmap.h:14-23, 340-373`

APPENDIX C

Rev0 bill of materials, principal parts

U1	RP2040, QFN-56	cartridge bus server, <code>fujivcs</code>
U2	W25Q16JVSSIQ	2 MB QSPI flash for the RP2040
U3, U4, U5	74LVC245APW	A0–A7, A8–A12 (fixed A→B) and D0–D7 (DIR on GP26) at 3.3 V
U6	ESP32-S3-WROOM-1-N16R8	FujiNet, <code>fujiversal-atari2600</code> ; USB host for U1
U7	CP2102N-A02-GQFN28	USB-C to UART0, S3 flashing and console
U8	UMH3N	esptool auto-program, DevKitC-1 style
U9	AP63203WU	3.3 V / 2 A buck
L1	SWPA4030S 6.8 μ H	buck inductor
D1	BAT54C	RESET steering: one button resets U1 (RUN) and U6 (EN)
D2	KT-0603G	RP2040 activity LED, GP25
D3	WS2812B-2020-V6	status LED on IO48; 3.3 V-rated part
D4–D6	ESD5Z5.0T1G	ESD on USB D+, D– and VBUS
D7, D8	SS34	console +5 V and VBUS OR-ing
J1	2×12 edge, 2.54 mm	the cartridge connector (original footprint)
J2	TF-015	microSD, push-push, SPI
J3	TYPE-C-31-M-12	USB-C
Y1	ABM8 12 MHz	RP2040 crystal, 15 pF loads
SW1–SW4	TL3342	RESET (both), BOOTSEL (RP), S3 EN, S3 BOOT
RN1	4×10 k	SD pull-ups

Table 34. Principal parts of Rev0. 78 parts are assembled, all on the component face. Every LCSC code was checked against stock. `Fujiversal-Atari2600-Rev0-BOM.csv`

APPENDIX D

FujiBus quick reference

END	\$C0	frame delimiter, sent before and after every frame
ESC	\$DB	escape prefix
ESC_END	\$DC	after ESC : a literal \$C0
ESC_ESC	\$DD	after ESC : a literal \$DB

Table 35. SLIP framing. `FujiBusPacket.h:16-21`

0	device	destination device id
1	command	command id; in a reply, ACK \$06 or NAK \$15
2-3	length	total decoded packet length including the header, little-endian
4	checksum	8-bit sum with end-around carry over the whole packet with this byte zeroed
5	descr	first field descriptor; \$00 if there are no parameters

Table 36. The six-byte header. `static_assert(sizeof(fujibus_header) == 6) . FujiBusPacket.cpp:17-26`

descr		
0	0	—
1-4	1-4	1 byte
5	1	2 bytes
6	2	2 bytes
7	1	4 bytes

Table 37. Field descriptors. Bit 7 means another descriptor byte follows. Values are written little-endian after all descriptors; whatever remains is the payload. Tables: `numFieldsTable = {0,1,2,3,4,1,2,1}` , `fieldSizeTable = {0,1,1,1,1,2,2,4}` .
`FujiBusPacket.cpp:28-35`

\$31 — \$3F	DISK	virtual disk drives
\$40 — \$43	PRINTER	printers
\$45	CLOCK	real-time clock
\$50 — \$53	SERIAL	modem / serial passthrough
\$70	FUJINET	the Fuji control device
\$71 — \$78	NETWORK	N1:-N8:
\$FF	DBC	the bus controller: the RP2040 itself, target of the ROM push

Table 38. Device ids. `include/fujiDeviceID.h`

APPENDIX E

Sources and glossary

fujinet-firmware	branch 2600-experiment , 7432186c0 , 2026-09-27	pico/atari-2600/ : firmware/ (RP2040), testrom/ (6502 clients and library), emu/ (MAME device, harnesses), host_test/ , tools/ , README-fujinet.md ; adapter side: lib/bus/rs232/ , lib/hardware/ACMChannel.* , lib/media/rs232/diskTypeROM.cpp , include/pinmap/fujiversal-atari2600.h , build-platforms/platformio-fujiversal- , atari2600.ini src/main.cpp , docs/fujiversal-flashing.md
fujinet-hardware	branch atari2600-rev0 , 84a8c09 , 2026-09-27	ATARI-2600/Fujiversal-Atari2600/ : README, BOM, KiCad project, docs/ renders and schematic
fujinet-config	branch add-atari-2600 , b4e3805 , 2026-09-13	atari-2600/ : CONFIG and its README
fujinet-battleship	9de5ef6 , 2026-09-16	atari2600/ : Battleship and its README
fujinet-manuals	atari-2600/programmers-handbook	the programmer's view; its MAME screens are reused here
MOS Technology	6500 family datasheet AC characteristics	tADS, tHA, tDSU, tMDS, tHW in Chapter 6

Table 39. Where every fact in this document comes from. Line numbers in the text refer to these revisions.

ACKSEQ	the sequence number the cartridge last answered; \$1F00 , painted last
arm / commit	the two stores that make one register write: \$1D00 +n, then the value to \$1DFF
arming gate	\$B5 to \$1DFC then \$4A to \$1DFD ; nothing on the control page decodes until it has arrived
claim	the four bytes FUJI at \$1F10 ; present in a client, absent in a game
DBC	device \$FF , the bus controller: the RP2040 as a FujiBus endpoint, target of the ROM push
fixed half	the last 2K of an image, served at \$1800 – \$1FFF from every bank, with the mailbox painted over its lower 1.75K
fixed tail	\$1F20 – \$1FFB , the 220 bytes of client code every bank reaches at the same address
FujiBus	the SLIP-framed packet protocol between the cartridge and the adapter, FEP-004
hotspot	an address whose access has a side effect: a bank switch, or on this cartridge a one-shot operation
mailbox	the whole console-facing protocol: control page, TX page, reply window, status page, text planes
plane	one of six 128-byte arrays of composed glyph bytes, one per GRP write of the kernel
slice	one 512-byte half of a reply of up to 1024 bytes
swap	replacing the served image with the staged one, on a store to \$1DFE after BOOTLOCK
TX stream	the parameters and payload of a request, appended one byte per store to page \$1E
window	the 4K of the 6507's address space with A12 high, \$1000 – \$1FFF

Table 40. Glossary.